



UT1 - M2 MIAGE 2IS
INNOVATIVE INFORMATION SYSTEMS

Internship Report

Design and Development of Just-In-Time Immersive Video Coaching to Support CPAP Adherence

Student: Pamit Duggal

Academic Supervisor: Prof. David Simoncini

Laboratory Supervisors: Prof. Nejib Moalla & Mme. Yasaman Kakaei Siahkal

27 August 2026

Acknowledgements

This internship was an incredible six-month journey, and I want to thank the people who made it possible.

At **Laboratoire DISP (UR4570)** in Lyon, I had the privilege of working alongside **Ms. Yasaman KAKAEI SIAHKAL**. Thank you, Yasaman, for your patience, technical insight, and day-to-day mentorship. Whether we were debugging edge inference latency, refining the 7-layer predictive pipeline, or preparing our paper for IEEE EMBS BSN 2026, your encouragement kept the project moving forward.

I am deeply grateful to **Professor Nejib MOALLA**. Thank you for welcoming me into the lab, trusting me with this research, and sharing your vision for distributed digital health platforms within the European 6G-Path trial. Your advice during our sprint reviews gave this project its direction.

I also want to thank the team at **Linde HomeCare France**. Having access to real, anonymized telemetry from thousands of patients grounded our models in practical clinical reality rather than hypothetical assumptions.

Back home in Toulouse, my sincere thanks go to **Professor David SIMONCINI**, **Professor Chihab HANACHI**, and **Dr. Umberto GRANDI**, as well as the entire **MIAGE 2IS faculty** at **Université Toulouse Capitole**. The coursework in distributed software, enterprise data systems, and decision support gave me the exact technical toolkit I needed to design and deliver this system.

Finally, to my family and friends: thank you for your unconditional love, steady encouragement, and for keeping me grounded throughout my studies.

Pamit Duggal
Lyon, August 2026

Abstract

Continuous Positive Airway Pressure (CPAP) works well for obstructive sleep apnea, but only if patients keep wearing the mask. In practice, nearly half quit within three months. They stop because of practical everyday friction: straps pinching, cold air blowing into their eyes, or dry throat from poor humidity. Standard telemonitoring uploads daily statistics to hospital servers, but nurses only review those logs weeks later, long after the patients have already discontinued daily therapy.

During my six-month internship at DISP Lab in Lyon, in collaboration with Linde HomeCare France, I built an edge-to-cloud system to fix this delay. Instead of waiting weeks, a bedside Raspberry Pi 5 runs an INT8-quantized TinyML model that classifies 20-second sensor windows in under 315 milliseconds. It pairs CPAP airflow logs with five consumer medical wearables (Withings ScanWatch, BPM Core, Masimo Rad-G, Hexoskin ProShirt, and SomnoArt) to pinpoint the exact mechanical or physiological issue. Raw sensor data never leaves the home. When an anomaly occurs, the system fetches a targeted 10-second coaching video from our server in under 1.4 seconds. For compound issues, telemetry routes to a dedicated AI Model VM running statistical drift detection, patient clustering, and survival analysis across 41,117 patients.

To stop generative video models from hallucinating medical metrics, we decoupled Python Matplotlib vector charts from Google Vertex AI Veo 2.0 background animations. A dedicated Video VM serves 27 Full HD coaching videos with bilingual English and French WebVTT subtitles. We demonstrated the platform live at the European 6G-Path trial in Berlin and published our findings in the **Proceedings of the IEEE EMBS International Conference on Body Sensor Networks (BSN 2026)**.

Keywords: Obstructive Sleep Apnea, CPAP Adherence, TinyML, Raspberry Pi 5, FastAPI, Dempster-Shafer Fusion, Survival Analysis, Generative AI, Google Vertex AI, WebVTT.

Contents

List of Figures	5
List of Tables	5
Glossary	6
1 Introduction	7
1.1 Context and Motivation	7
1.2 Mission and Objectives	7
1.3 Key Deliverables	8
1.4 Report Structure	9
2 Host Structure	9
2.1 Laboratoire DISP (UR4570), Université Lumière Lyon 2	9
2.2 Linde HomeCare France	9
3 Problem Statement and Needs Analysis	10
3.1 Clinical Signals and Wearable Biomarkers	10
3.2 Why Existing Solutions Fail	11
3.3 System Requirements	11
4 Background and Theoretical Foundations	12
4.1 Edge Computing and INT8 Quantization	12
4.2 The 7-Layer Clinical AI Model Architecture	12
4.3 Technology Stack Justification: Why These Libraries and APIs?	13
4.3.1 FastAPI vs. Flask and Django	14
4.3.2 TensorFlow Lite INT8 vs. Full-Precision PyTorch	14
4.3.3 Decoupled WebVTT vs. Burned-In Video Subtitles	14
4.3.4 OpenCV vs. Subprocess FFmpeg Spawning	15
5 Phase 1: Edge-to-Cloud Prototype	15
5.1 Data Pipeline and SQLite Normalization	15
5.2 Edge Ingestion Setup	16
5.3 Execution Pathways: Fast Path vs. Slow Path	17
5.3.1 Fast Path: Bedside Real-Time Sensor Triage (Under 1 Minute)	17
5.3.2 Slow Path: Cloud Multi-Day Retrospective Predictive Modeling (AI Model VM)	18
5.3.3 Distinction: Ingestion Pathways vs. Video Delivery Scenarios	18
5.4 Rule-Based Edge Classifier	19
5.5 Decoupled Video Synthesis: Solving AI Hallucinations	19
6 Phase 2: Scaling to a 27-Video Metadata-Driven Library	20
6.1 Expanding the Library to 27 Videos	20
6.1.1 Google Veo Video Generation and Clinical Prompt Engineering	21
6.2 Three Delivery Scenarios	22
6.2.1 Dual-Language WebVTT Subtitle Architecture and Naming Conventions	23
6.3 Automated Metadata Generation	24
6.4 Video VM Server Microservice Operations	25
7 Phase 3: AI Model VM Model as a Dedicated Server	27
7.1 Turning the Research Notebook into a Live Service	27
7.2 Middleware Interception and Runtime Compatibility Hooks	28
8 Project Management & Engineering Methodology	31
8.1 Agile Scrum Framework	31
8.2 Timeline and Deliverables	31
9 Innovation	32
9.1 Algorithmic Innovation: Closing the Loop with Multi-Sensor Fusion	32
9.1.1 Multimodal Sensor Disambiguation	32

9.1.2	Dempster-Shafer Evidence Combination with Conflict Handling	33
9.2	Architectural Innovation: Decoupled Video Synthesis for Clinical Accuracy	33
9.2.1	Separating Deterministic Charts from Visual Diffusion	33
9.3	Production Middleware Innovation: Runtime Interception via <code>api_data_loader.py</code>	34
9.3.1	Transparent File I/O Interception	34
9.4	Embedded Optimization Innovation: Sub-315ms Local Edge Triage	34
9.4.1	Quantized Inference on the Raspberry Pi 5	34
9.5	Collaborative and Engineering Methods	35
10	Final Consolidated System Architecture	35
10.1	Global Distributed Topology and End-to-End Flow	35
10.2	Network Endpoints	37
10.3	Security and API Keys	38
11	Results, Benchmarks & Experimental Evaluation	38
11.1	AI Model VM Performance (Cohort of 41,117 Patients)	38
11.2	Video VM Server Performance	40
11.3	Raspberry Pi 5 Edge Telemetry Benchmarks	40
11.4	Summary of Experimental Results	41
12	Project Assessment, Competency Acquisition, and Conclusion	42
12.1	Synthesis of Tangible Deliverables	42
12.2	Technical Skills Acquired and Deepened	42
12.3	Professional and Interpersonal Growth	43
12.4	Direct Alignment with Master 2 MIAGE Coursework	43
12.5	Critical Reflections and Lessons Learned	44
12.6	Perspectives for Future Work	44
12.7	Conclusion	44
12.7.1	Summary of Empirical Findings and Benchmarks	44
12.7.2	Engineering Takeaways and Impact	45
	Bibliography	46
	References	46
A	Comprehensive 27-Video Clinical Registry	47
B	Accepted IEEE EMBS BSN 2026 Conference Paper	48

List of Figures

1.1	Clinical Coaching Knowledge Base Word Cloud	8
3.1	Multimodal Wearable Biomarker Devices	10
3.2	Multimodal Telemetry and Biomarker Cross-Correlation Heatmap	11
5.1	Relational Schema for CPAP Telemetry and Clinical Intervention Tables	15
5.2	Raspberry Pi 5 FastAPI Edge Service Console	17
5.3	Telemetry Ingestion: Fast Path vs. Slow Path Flow	19
5.4	Deterministic Dark-Mode Clinical Alert Charts	20
5.5	Decoupled Generative AI Prompt-to-Video Synthesis Workflow	20
6.1	Comprehensive 27-Module Clinical Coaching Video Library Grid	22
6.2	Video VM: Three Coaching Delivery Modes	23
6.3	Dual-Language HTML5 WebVTT Subtitle Registry (54 Files)	24
6.4	Keyframe Screenshots from AI-Synthesized Coaching Videos	24
6.5	Video VM Microservice Runtime Operations (Port 8080)	26
7.1	7-Layer AI Model VM Predictive Pipeline Architecture	27
7.2	PCA Feature Space and MeanShift Clustering for Patient Adherence Phenotyping	28
7.3	AI Model VM Server Runtime Operations (Port 8000)	30
10.1	Global Consolidated Distributed System Architecture	36
10.2	Web Application Catalog Interface for the 27-Video Coaching Modules	37
10.3	Patient Coaching Web Player with Synchronized Dual-Language Subtitles	37
11.1	National Cohort Compliance Breakdown ($N = 41, 117$) and Top SHAP Predictors	39
11.2	CUSUM and EWMA Control Charts for Adherence Decline Detection	39
11.3	End-to-End Latency Budget vs. Observed Execution Times	41

List of Tables

1	Glossary of technical, medical, and architectural terms	6
3.1	System Requirements and Engineering Constraints	12
4.1	Architectural and Library Selection Trade-offs	14
6.1	The Three Video Delivery Scenarios	23
8.1	Internship Timeline and Major Milestones	31
8.2	Technical Challenges and Engineered Solutions	32
11.1	AI Model VM Server Metrics across 41,117 Patients (CPAP_AI_SERVER_KPIS.txt)	38
11.2	Video VM Microservice Performance (CPAP_VIDEO_SERVER_KPIS.txt)	40
11.3	Real-Time Edge Detection Latencies on Raspberry Pi 5	40
A.1	Complete 27-Video Clinical Coaching Registry	47

Glossary

Term / Acronym	Meaning
AHI	Apnea-Hypopnea Index (number of breathing pauses per hour of sleep)
API	Application Programming Interface
BPM Core	Withings blood pressure monitor with digital ECG and stethoscope
BSN	Body Sensor Networks (IEEE EMBS International Conference)
CAMARA	Linux Foundation open API project for telecommunication networks
CIFRE	French industrial research training agreement (<i>Convention Industrielle de Formation par la Recherche</i>)
CPAP	Continuous Positive Airway Pressure
CUSUM	Cumulative Sum statistical control chart
DISP	<i>Décision et Information pour les Systèmes de Production</i> (UR4570 Laboratory)
ECE	Expected Calibration Error
ESS	Epworth Sleepiness Scale
ETL	Extract, Transform, Load data pipeline
EWMA	Exponentially Weighted Moving Average
FastAPI	Python framework for building asynchronous web APIs
HRV	Heart Rate Variability (RMSSD, SDNN)
IoT	Internet of Things
MEC	Multi-access Edge Computing
MLP	Multi-Layer Perceptron
OSA	Obstructive Sleep Apnea
PSQI	Pittsburgh Sleep Quality Index
QoD	Quality on Demand (5G network API)
REST	Representational State Transfer
SCP	Secure Copy Protocol
SHAP	SHapley Additive exPlanations
SpO2	Blood oxygen saturation percentage
TFLite	TensorFlow Lite
TTD	Time-to-Dropout (survival duration)
VTT	Web Video Text Tracks (HTML5 subtitle format)

Table 1: Glossary of technical, medical, and architectural terms

1. Introduction

1.1 Context and Motivation

Obstructive sleep apnea (OSA) is straightforward in theory, but exhausting in practice. During sleep, upper airway muscles relax, blocking airflow dozens of times every hour. Patients wake up gasping, their blood oxygen plummets, and cardiovascular strain builds up over months. The standard prescription is Continuous Positive Airway Pressure (CPAP). A bedside blower forces room air through a silicone mask, physically splinting the airway open.

When patients wear their mask for at least four hours every night, CPAP works. Daytime fog clears. Blood pressure drops. But sleeping with a pressurized mask strapped to your face is uncomfortable. In practice, between 30% and 50% of patients give up within their first three months.

So why do patients give up on therapy? It's almost never a lack of belief in the treatment itself. What actually happens is a slow accumulation of physical discomfort: straps that press too hard against the cheekbones, a mask seal that shifts when the patient rolls onto their side and lets cold air in, or dry, irritated airways because the humidifier settings were never adjusted.

Current telemonitoring in France fails to catch these early hurdles. CPAP machines log summary usage and dump them once every 24 hours to hospital servers. Those logs sit in databases until care teams review monthly summaries. By the time anyone notices, the patient has already given up on the therapy.

During my six-month internship at **Laboratoire DISP** with **Linde HomeCare France**, I set out to break that cycle. We evaluate sensor data on a bedside Raspberry Pi the minute the patient wakes up, diagnose the exact mechanical annoyance, and send a short, ten-second coaching clip to the patient's phone first thing in the morning.

1.2 Mission and Objectives

My main task was taking the clinical coaching concepts formulated by my supervisor, Ms. Yasaman Kakaei Siahkal, and building a working edge-to-cloud software pipeline around them.

Analyzing the transcripts of our 27 coaching topics showed clear patterns in what patients struggle with most: mask strap tightness, humidifier levels, ramp settings, and sleep fragmentation.

Figure 1.1 shows the word cloud extracted from the 27 clinical coaching scripts, which identifies the most frequent trouble spots.

4. A live demonstration at the 6G-Path trial in Berlin in June 2026.
5. An accepted conference paper in the **Proceedings of IEEE EMBS BSN 2026**.

1.4 Report Structure

Chapter 2 introduces DISP Lab and Linde HomeCare France. Chapter 3 defines the clinical problem and engineering requirements. Chapter 4 presents the mathematical foundations behind the anomaly detection and survival models. Chapters 5, 6, and 7 describe the three implementation phases. Chapter 8 reviews our sprint methodology and technical debugging. Chapter 9 provides an in-depth analysis of our algorithmic, architectural, and methodological innovations. Chapter 10 details the final distributed network architecture. Chapter 11 presents benchmark data across the 41,117-patient cohort. Finally, Chapter 12 assesses the built deliverables, technical and soft skills acquired, direct alignment with the UT1 MIAGE 2IS curriculum, and synthesizes the overall project conclusion.

2. Host Structure

2.1 Laboratoire DISP (UR4570), Université Lumière Lyon 2

The **DISP Laboratory** (Decision and Information Systems for Production Systems) brings together researchers from Université Lumière Lyon 2, INSA Lyon, and Université Jean Monnet Saint-Étienne. The lab specializes in distributed information systems, supply-chain logistics, and digital healthcare platforms.

I carried out my research under **Ms. Yasaman KAKAEI SIAHKAL** and **Prof. Nejib MOALLA**, working within the framework of the European 6G-Path consortium. The consortium looks at how next-generation telecommunication capabilities (such as 5G network slicing and multi-access edge computing) can make remote healthcare applications more responsive.

2.2 Linde HomeCare France

Linde HomeCare France is a major home healthcare provider in France, managing respiratory support for tens of thousands of patients with obstructive sleep apnea, COPD, and chronic respiratory failure.

In everyday operations, Linde HomeCare:

- Delivers, configures, and services CPAP machines in patients' homes.
- Sends respiratory technicians and nurses for initial mask fittings and follow-up adjustments.
- Monitors nightly usage to track patient therapy adherence and treatment efficacy.

Linde provided the anonymized CPAP datasets, clinical coaching rules, and domain expertise that grounded our models in real-world clinical practice.

3. Problem Statement and Needs Analysis

3.1 Clinical Signals and Wearable Biomarkers

Four hours is the critical benchmark in sleep medicine. If a patient logs at least four hours of nightly mask wear on 70% of nights, French social security reimburses the machine. If usage slips below that line, they lose coverage and symptoms return. To catch adherence drift early, our system tracks five core telemetry streams:

1. **Usage Duration:** Nightly hours of active mask pressure (`Use`).
2. **Mask Leakage:** Unintentional airflow leakage in L/min (`Leaks95`, `Leaks90`, or `Leaks0`).
3. **Apnea-Hypopnea Index (AHI):** Residual airway collapses per hour of sleep.
4. **Therapeutic Pressure:** 90th percentile pressure delivered in cmH₂O (`Pressure90`).
5. **Wearable Biomarkers:** Nocturnal SpO₂ and HRV from a Withings ScanWatch, blood pressure and ECG from a Withings BPM Core, pulse oximetry from a Masimo Rad-G, respiratory effort from a Hexoskin ProShirt, and sleep staging from SomnoArt.

To capture physiological factors beyond basic airflow, standard CPAP telemetry is integrated with five commercial medical wearables. This multi-sensor framework collects continuous pulse oximetry, single-lead ECG, blood pressure, dual-band respiratory effort, and digital sleep architecture stages to contextualize nocturnal mask removals.

Figure 3.1 illustrates the five commercial wearable devices deployed alongside the CPAP unit to provide multimodal physiological surveillance.

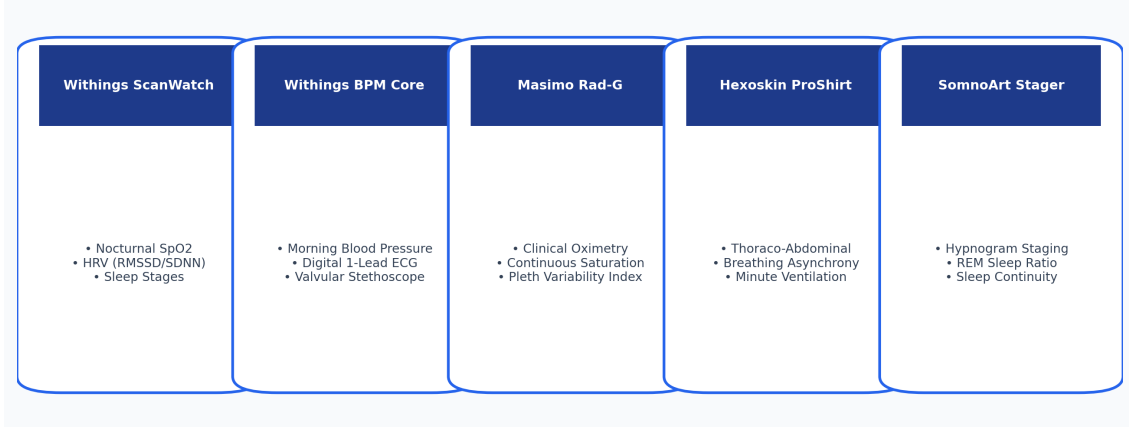


Figure 3.1: Multimodal Wearable Biomarker Devices

Statistical cross-correlation analysis across the multi-sensor dataset reveals clear dependencies between airflow metrics and physiological stability. In particular, elevated mask leakage (≥ 24 L/min) strongly correlates with nocturnal desaturation episodes ($\text{SpO}_2 < 90\%$) and micro-arousals. This shows that mechanical mask leaks directly cause oxygen drops and disrupt sleep.

Figure 3.2 presents the cross-correlation matrix heatmap mapping dependencies across CPAP airflow metrics, pulse oximetry, blood pressure, and sleep staging.

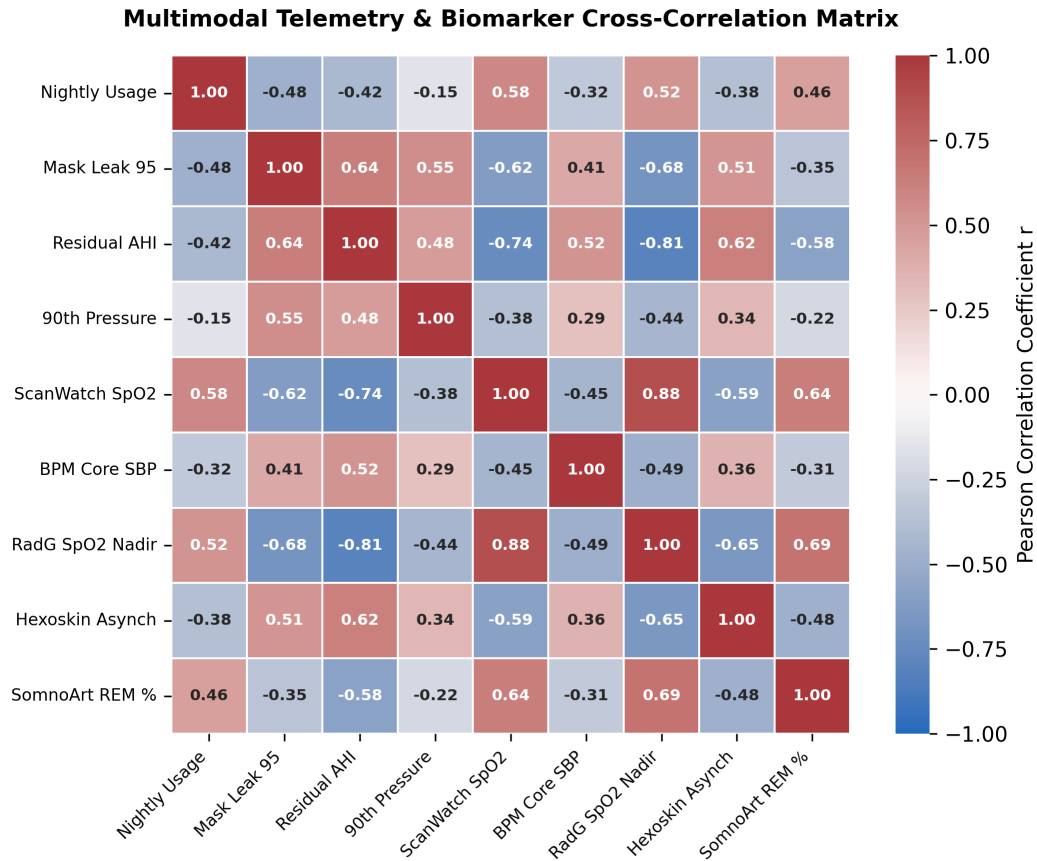


Figure 3.2: Multimodal Telemetry and Biomarker Cross-Correlation Heatmap

3.2 Why Existing Solutions Fail

Current CPAP telemonitoring fails for three practical reasons:

1. **Slow Feedback Loops:** Telemetry data is treated like monthly billing records. Numbers sit on remote servers until a nurse runs quarterly compliance filters. By that point, the patient has usually already stopped using the machine.
2. **Generic Paper Handouts:** A static flyer cannot tell a patient whether their mask is leaking because the bottom strap is too loose or because the cushion is worn out. Patients need immediate, visual feedback on the exact problem they experienced that night.
3. **Single-Modality Blindness:** CPAP flow sensors only measure air moving through the hose. The sensors alone can't distinguish whether a patient pulled off their mask during the night because of nasal congestion, because the straps were uncomfortable, or because of a genuine cardiac event. Without wearable physiological context, clinicians see only low usage numbers without the underlying cause.

3.3 System Requirements

To ensure that automated coaching is clinically effective, secure, and delivered when patients need it most, the distributed system must satisfy strict operational constraints. In particular, the edge runtime must protect patient telemetry by classifying windows locally without streaming raw sensor

data across public networks, ensure zero medical hallucinations in video synthesis, and meet a strict 60-second delivery window upon morning wake-up.

Table 3.1 summarizes these core system requirements alongside their engineering rationales.

Table 3.1: System Requirements and Engineering Constraints

Requirement	Engineering Rationale
Fast Edge Processing	The Raspberry Pi 5 must process sensor windows locally to preserve patient privacy and produce alerts immediately after waking.
Zero Medical Hallucination	Generative AI must never fabricate medical numbers. Graphs must come directly from verified sensor data.
Bilingual Subtitles	Every coaching clip must have synchronized English and French HTML5 WebVTT subtitles.
Separated AI Server	The 7-layer AI Model VM predictive pipeline (<code>M4_FINAL_F2.ipynb</code>) must run on its own port (8000) so retraining does not affect video streaming on Port 8080.
Strict Delivery Window	Video coaching must arrive on the patient dashboard within 60 seconds of waking.

4. Background and Theoretical Foundations

4.1 Edge Computing and INT8 Quantization

Sending raw data over public cellular networks is slow, costly, and raises real privacy concerns, so we ruled that out early. Instead, triage happens locally, right at the bedside, on the Raspberry Pi 5. The data never leaves the device and only small, anonymized JSON alerts get sent out when something looks off.

Running neural networks on an embedded ARM Cortex-A76 CPU requires discipline. Our first full-precision PyTorch model consumed over 800 MB of RAM and drove the CPU to thermal throttling within fifteen minutes. To fix this, I quantized the weights down to 8-bit integers using TensorFlow Lite Micro. The quantization maps continuous floating-point weights W_f to int8 integers q :

$$q = \text{round} \left(\frac{W_f}{S} \right) + Z \quad (4.1)$$

where S is the scaling factor and Z is the integer zero-point offset.

On our physical test bench in Lyon, quantization cut the model footprint from 2.4 MB down to 486 kB. Inference latency dropped to 263 to 313 ms per 20-second window, leaving the Pi’s CPU free for background network tasks.

4.2 The 7-Layer Clinical AI Model Architecture

My supervisor, Ms. Yasaman Kakaei Siahkal, developed the predictive pipeline inside a research notebook (`M4_FINAL_F2.ipynb`). My task was to turn her mathematical framework into a live, standalone service that could process patient data around the clock. The architecture breaks into seven distinct stages:

- **Layer 0: Statistical Process Control (Sensor Drift):** Calculates each patient’s baseline mean and variance (μ_0, σ_0) during their first 56 nights. It runs Cumulative Sum (CUSUM) charts

and Exponentially Weighted Moving Average (EWMA) filters ($\lambda = 0.2$) so that a temporary bad night (like a cold) does not trigger a false alarm.

- **Layer 1: Patient Phenotyping:** Projects usage metrics into principal component space and applies MeanShift clustering to group patients into distinct behavioral profiles.
- **Layer 2: Multimodal Evidence Fusion:** Combines independent diagnostic metrics across four channels (*CPAP*, *Care*, *Survey*, and *Biomarker*) using Dempster-Shafer belief functions. If sensors contradict each other with a conflict score $K \geq 0.70$, the system flags the event for manual clinician review.
- **Layer 3: Survival Analysis and Risk Stratification:** Predicts two separate dropout risks: voluntary therapy abandonment using LightGBM and administrative contract cancellation using Cox Proportional Hazards regression.
- **Layer 4: Counterfactual Intervention Recommendation:** Evaluates simulated 30-day adherence gains across possible actions (Home Visit, Phone Call, SMS, Video Coaching) to select the most effective intervention for each patient.
- **Layer 5: Wearable Biomarker Anomaly Detection:** Monitors nightly wearable signals (Withings, Masimo, Hexoskin, SomnoArt) for acute health events like hypoxia, irregular heart rhythms, or severe sleep fragmentation.
- **Layer 6: Action Plan Formulation:** Packages the calculated risk tier, clinical rationale, and assigned video module into an authenticated JSON payload for dashboard dispatch.

4.3 Technology Stack Justification: Why These Libraries and APIs?

Every framework, library, and communication protocol in this architecture was chosen after weighing practical trade-offs against common alternatives. Key design decisions include selecting FastAPI for asynchronous non-blocking request handling, quantizing models to INT8 TFLite Micro to operate within a 500 kB RAM budget on the ARM Cortex-A76 processor, and adopting standalone WebVTT subtitle files to prevent doubling our video storage footprint.

Table 4.1 details these architectural selections, comparing chosen technologies against rejected alternatives.

Table 4.1: Architectural and Library Selection Trade-offs

Component	Selected Technology	Rejected Alternative	Engineering Rationale & Justification
Web API Framework	FastAPI (ASGI)	Flask / Django (WSGI)	Asynchronous non-blocking I/O handles concurrent dashboard polls, Pydantic ensures type safety, and latency remains under 10ms on ARM Pi.
Edge Inference	TFLite Micro (INT8)	PyTorch / ONNX Runtime	8-bit integer quantization drops RAM footprint under 500 kB with zero floating-point latency on Cortex-A76.
Audio Transcription	OpenAI Whisper	Cloud STT (Google/AWS)	Runs locally for patient privacy with zero API egress cost, accurately parsing French and English medical terms into WebVTT.
Keyframe Extraction	OpenCV (cv2)	MoviePy / FFmpeg CLI	Decodes C++ frames directly to NumPy arrays without spawning heavy subprocesses or leaking memory.
Subtitle System	HTML5 WebVTT	Burned-in Hardcoded Text	Allows instant English and French switching without re-rendering 241 MB video files, and remains accessible for screen readers.
Video Synthesis	Python + Vertex AI	End-to-End AI (Runway)	Decouples exact Python Matplotlib charts from Veo animation to eliminate hallucinated medical numbers.
5G Network Slice	CAMARA QoD API	Best-Effort Public 4G	Reserves guaranteed radio bearer bandwidth to meet the 60-second video delivery SLA during morning network peaks.
Edge Security	SlowAPI + HMAC	Heavy Nginx Gateway	Protects the low-power Pi 5 against brute-force polling and crawler flooding with zero added infrastructure overhead.

4.3.1 FastAPI vs. Flask and Django

We picked FastAPI for all three service nodes (Edge, AI Server, Video VM) over WSGI frameworks like Flask or Django. Flask handles requests synchronously by default. A heavy video stream or slow database query can freeze incoming polling requests unless you spin up multiple worker processes, which eats up scarce RAM on the Raspberry Pi. FastAPI runs on the asynchronous Uvicorn ASGI server. It handles concurrent dashboard queries and telemetry uploads on a single lightweight event loop with sub-10ms response times. Pydantic models validate incoming JSON payloads automatically, catching malformed data before it hits route handlers.

4.3.2 TensorFlow Lite INT8 vs. Full-Precision PyTorch

Unquantized 32-bit floating-point PyTorch models consumed significant amount of RAM on the Pi, and the floating-point math was enough to slow the CPU down noticeably. TensorFlow Lite Micro with post-training INT8 quantization compresses weight matrices into 8-bit integers. On the Pi’s ARM Cortex-A76 core, integer math taps into NEON SIMD instructions. That keeps RAM usage below 500 kB and inference latency under 315 ms per 20-second sensor window.

4.3.3 Decoupled WebVTT vs. Burned-In Video Subtitles

Burning subtitles directly into video pixels requires rendering and storing two separate 1080p MP4 files for every clinical topic: one for English, one for French. That would double our video storage

footprint from 241 MB to nearly 500 MB and prevent patients from switching languages on the fly. Using standalone Web Video Text Tracks (.vtt) keeps video files language-agnostic. It lets patients switch languages instantly in HTML5 video players and allows screen-readers to parse the advice.

4.3.4 OpenCV vs. Subprocess Ffmpeg Spawning

To index the 27 videos, we extract thumbnail keyframes at 1.5s, 5.0s, and 8.5s. Spawning external ffmpeg command-line processes from Python introduces process-creation overhead and risks leaving zombie processes behind on failure. Using OpenCV’s native C++ bindings (cv2.VideoCapture) allows our indexing script to seek directly to target timestamps, grab frame buffers into NumPy arrays, and write optimized JPEG thumbnails with zero subprocess overhead.

5. Phase 1: Edge-to-Cloud Prototype

5.1 Data Pipeline and SQLite Normalization

In practice, raw telemetry exports from Linde HomeCare arrived with significant format inconsistencies. Different CPAP brands formatted output fields differently: ResMed machines exported leak rates under proprietary headers, SEFAM hardware used separate naming conventions, and Philips devices formatted dates with alternating delimiters. In some tables, test patient accounts (999999xxx) were mixed in with production cohort logs.

I wrote an automated Python and SQLite ETL pipeline to clean and normalize these exports. The script parses all raw files into six relational tables: Usage3, Monitoring3, Interventions, Patients all, Treatment periods, and Mask assignments. Every record maps to Linde’s permanent patient identifier (AtHomePatientId, such as 191831).

This schema separates long-term usage totals from daily sensor feeds and recorded clinical interventions. Splitting the tables this way keeps query times low when our models score patient records in production.

Figure 5.1 shows the normalized relational entity-relationship schema designed for CPAP telemetry and clinical intervention records.

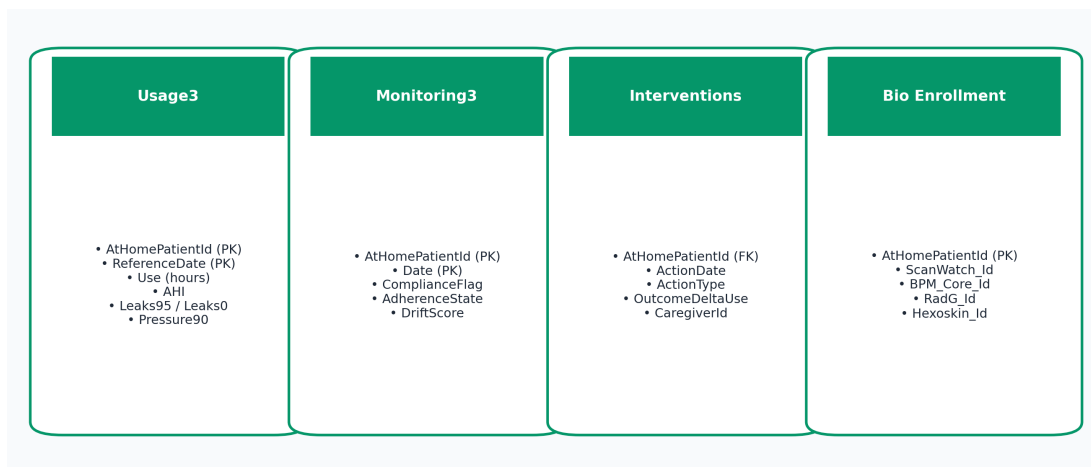


Figure 5.1: Relational Schema for CPAP Telemetry and Clinical Intervention Tables

5.2 Edge Ingestion Setup

We configured the Raspberry Pi with an SSH daemon and an incoming drop directory (`/data/incoming`) using public-key authentication. Hardware deliveries were delayed during my first month, stalling physical bench testing. Instead of waiting, I set up an ARM64 Raspberry Pi virtual machine in VMware on my Windows laptop. That allowed me to write and debug all bash ingestion scripts, SSH key handshakes, and SCP transfers weeks before the physical Pi 5 board arrived in Lyon.

The core edge telemetry ingestion and classification logic is implemented in `edge_detection.py`. The function `evaluate_edge_telemetry` extracts sliding 20-second sensor readings, checks them against clinical safety thresholds, and maps identified anomalies directly to the appropriate video module identifier before pushing a lightweight JSON alert.

```
# Source File: code Raspberry pi/edge_detection.py
def evaluate_edge_telemetry(row: dict) -> dict:
    leak = _num(row, "Leaks95", default=0.0)
    ahi = _num(row, "AHI", default=0.0)
    usage = _num(row, "Use", default=0.0)

    # Check for excessive air leakage around mask cushion
    if leak >= LEAK_CRITICAL:
        return {"trigger": "mask_leak_critical", "video_id": 14, "priority": "HIGH"}
    elif leak >= LEAK_MODERATE:
        return {"trigger": "mask_leak_moderate", "video_id": 1, "priority": "MEDIUM"}

    # Check for acute spikes in residual apnea events
    if ahi >= AHI_CRITICAL:
        return {"trigger": "severe_apnea_alert", "video_id": 15, "priority": "URGENT"}
    elif ahi >= AHI_ELEVATED:
        return {"trigger": "elevated_ahi_check", "video_id": 12, "priority": "HIGH"}

    # Triage sub-therapeutic nightly duration
    if usage < USAGE_FULL:
        return {"trigger": "low_usage_ramp", "video_id": 4, "priority": "MEDIUM"}

    return {"trigger": "routine_monitoring", "video_id": 3, "priority": "LOW"}
```

Listing 5.1: Core Edge Telemetry Triage Logic in `edge_detection.py`

As we can observe in the live terminal session in Figure 5.2, the Raspberry Pi 5 edge service boots a FastAPI server on port 8000, loads the quantized INT8 TensorFlow Lite model into local RAM, and executes this triage logic to classify incoming windows locally in under 315 ms without transmitting unencrypted raw medical data over the internet.

```

INFO: 100.84.123.41:0 - "POST /timing/c6951da3-f1ae-43ef-85dd-555e0f2dac82 HTTP/1.1" 200 OK
ACINFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [13660]
(.venv) raspberrypcap@raspberrypcap:~/Desktop/CPAP_Edge_copy $ skill -f "uvicorn app:app"; cd ~/Desktop/CPAP_Edge_copy && .venv/bin/uvicorn app:app --host 0.0.0.0 --port 8000
Library Loader: Successfully indexed 27 video metadata records.
INFO: Started server process [14290]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

=====
Δ CPAP / BIOMARKER ANOMALY DETECTED ON EDGE NODE
=====
Patient ID      : 999999021
Primary Trigger : nasal_congestion_relief
Severity Level  : LOW
Metrics Captured : {
  "leak": 12.0,
  "pressure": 0.0
}
Total Signals Flagged: 1
=====
[EDGE -> VIDEO VM] Auto-forwarding decision to http://159.84.143.246:8080/api/orchestrate for patient 999999021...
[EDGE -> VIDEO VM] Successfully orchestrated! Push status: success

=====
VIDEO COACHING SELECTION & DASHBOARD PUSH SUMMARY
=====
Patient ID      : 999999021
Scenario Executed : scenario_1_single
Video Asset Selected : Nasal Congestion Relief
* P1 Processing Time : 297.19 ms
=====

INFO: 100.84.123.41:0 - "POST /ingest HTTP/1.1" 200 OK

=====
◊ TRANSACTION TIMING (measured round trip - no clock skew)
=====
Patient ID      : 999999021
Round Trip Measured : 519.12 ms
  Phone -> P1 Transit : 110.06 ms
  * P1 Processing Time : 297.20 ms
Total Phone -> Ready : 408.16 ms
◊ 60-Second Budget : 0.408s / 60.0s (PASSED ✓)
=====

```

Figure 5.2: Raspberry Pi 5 FastAPI edge service console running real-time telemetry ingestion, TFLite inference, and local video selection on port 8000.

5.3 Execution Pathways: Fast Path vs. Slow Path

To understand how telemetry moves through the system, we distinguish between two distinct execution and ingestion pathways: the real-time edge **Fast Path** and the longitudinal cloud **Slow Path**. These two data ingestion pathways operate independently from the three coaching video delivery scenarios (Retrieve, Adapt, Generate) described in Chapter 6, and both pathways can trigger any of the three video scenarios on the Video VM server.

5.3.1 Fast Path: Bedside Real-Time Sensor Triage (Under 1 Minute)

The Fast Path handles immediate, acute therapy difficulties right at the patient's bedside:

1. **Direct Edge Ingestion:** Raw telemetry streams directly from physical bedside devices (the CPAP machine and paired wearables: Withings ScanWatch, Withings BPM Core, Masimo Rad-G, Hexoskin ProShirt, and SomnoArt) over Bluetooth Low Energy and USB into the bedside Raspberry Pi 5.
2. **Local TinyML Classification:** A Python ingestion script (`edge_detection.py`) processes sliding 20-second sensor windows in real time, classifying events with an INT8-quantized model in under 315 ms without uploading raw physiological data to external networks.
3. **Video Dispatch:** Upon confirming an anomaly, the Raspberry Pi 5 sends a lightweight JSON alert to the Video VM server (Port 8080).
4. **Immediate Playback:** The Video VM identifies or compiles the appropriate coaching media (via Scenario 1, 2, or 3) and pushes the payload to the patient dashboard frontend. The entire cycle completes in approximately 1.4 seconds.

5.3.2 Slow Path: Cloud Multi-Day Retrospective Predictive Modeling (AI Model VM)

The Slow Path analyzes longitudinal behavioral patterns and cumulative clinical deterioration over multiple nights of therapy:

1. **Direct Cloud Telemetry Sync (Every 2 to 3 Days):** Sensor devices and CPAP modems sync aggregated multi-day usage records directly to the Central Healthcare Database (Port 80) every 2 to 3 days. **Note:** This central backend ingestion operates independently and does *not* route through or rely upon the bedside Raspberry Pi 5.
2. **REST API Ingestion:** The AI Model VM (Port 8000) fetches these multi-day patient records over authenticated REST APIs using custom runtime middleware (`api_data_loader.py`).
3. **7-Layer Predictive Modeling:** The AI Model VM executes my supervisor’s research notebook (`M4_FINAL_F2.ipynb`), analyzing statistical process control drift (CUSUM/EWMA), unsupervised behavioral phenotypes (MeanShift), multi-source belief fusion (Dempster-Shafer), and 90-day dropout risk (dual-process survival modeling).
4. **Video Trigger Dispatch:** Once an intervention recommendation is computed, the integration middleware dispatches a video trigger to the Video VM server (Port 8080), which executes the matching media scenario and pushes the coaching package to the patient dashboard.

5.3.3 Distinction: Ingestion Pathways vs. Video Delivery Scenarios

It is essential to maintain a clear distinction between the two data ingestion pipelines and the three media generation modes:

- **Ingestion Pathways (How Data Flows):**
 - **Fast Path:** Direct Bluetooth/USB sensor ingestion into the Raspberry Pi 5 for real-time edge triage (< 1.4 s).
 - **Slow Path:** Direct 2 to 3 day synced telemetry into the Central Backend Database (Port 80), evaluated by the AI Model VM (Port 8000) using the 7-layer predictive notebook.
- **Video Delivery Scenarios (How Media is Assembled on Video VM):**
 - **Scenario 1 (Retrieve):** Direct playback of a single existing clip from the 27-video library.
 - **Scenario 2 (Adapt):** Dynamic server-side stitching of two or more clips with 1.5s cross-fades for compound triggers.
 - **Scenario 3 (Generate):** Synthesis of a completely new video using Google Vertex AI Veo 2.0 with decoupled Python Matplotlib charts and OpenAI Whisper WebVTT subtitles.
- **Universal Media Access:** Both the Fast Path edge node and the Slow Path AI Model VM can request any of the three video scenarios from the Video VM based on clinical urgency and anomaly complexity.

Figure 5.3 illustrates the end-to-end modular flow contrasting the edge-based Fast Path against the cloud-based Slow Path.

Telemetry Ingestion: Fast Path vs. Slow Path Flow

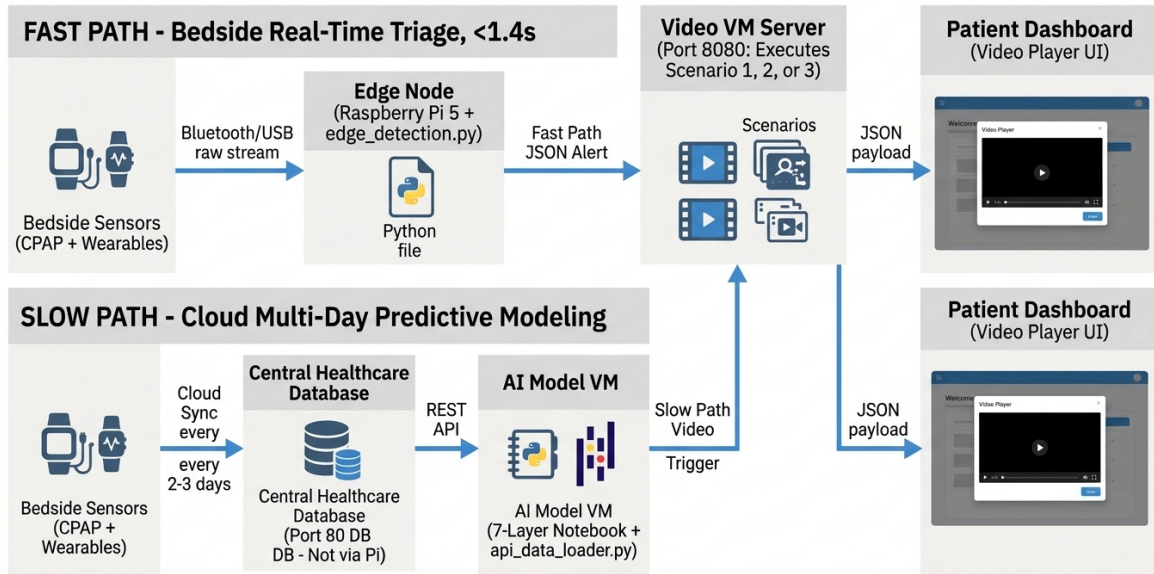


Figure 5.3: Modular end-to-end data flow contrasting the real-time edge Fast Path (bedside sensors to dashboard playback in < 1.4 s) against the cloud Slow Path (2 to 3 day synced logs through the 7-layer predictive notebook).

5.4 Rule-Based Edge Classifier

The edge detector evaluates nightly metrics against verified clinical thresholds:

- Mask Leak ≥ 24 L/min $\rightarrow$ Recommend strap tightening and seal check.
- Residual AHI ≥ 15 /hr $\rightarrow$ Escalate to medical provider.
- Usage < 4.0 h/night $\rightarrow$ Suggest daytime acclimation or pressure ramp mode.

5.5 Decoupled Video Synthesis: Solving AI Hallucinations

Initial exploratory tests using end-to-end generative video models (such as Luma AI and Google Veo) revealed significant reliability limitations for clinical interfaces. While character animations rendered smoothly, generative text within synthetic video frames suffered from visual distortions, warped chart axes, and fabricated numerical metrics. In medical decision support, unverified synthetic figures present substantial clinical safety risks.

I resolved this by decoupling video generation into two independent layers:

1. A Python Matplotlib script generates exact, high-resolution dark-mode charts directly from the patient's sensor data (Figure 5.4).
2. Google Veo is used solely for background character and equipment animations where exact quantitative text is not embedded in the video pixels.
3. Bilingual HTML5 WebVTT subtitle files (`.en.vtt`, `.fr.vtt`) handle text and speech overlays separately.

To guarantee medical accuracy in patient feedback, all clinical charts are rendered deterministically using Python Matplotlib. This eliminates the risk of AI diffusion models generating distorted axes or

fabricated numbers. Patients and clinicians see verified, reliable graphs of mask leakage and residual apnea trends.

Figure 5.4 displays the deterministic dark-mode clinical charts generated directly from patient telemetry records.

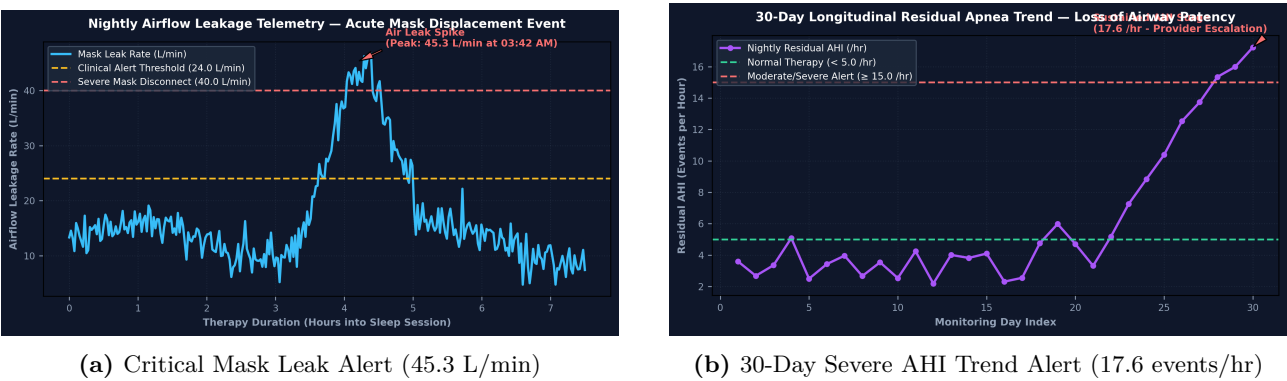


Figure 5.4: Deterministic Dark-Mode Clinical Alert Charts

The decoupled generative AI architecture separates clinical data rendering from visual video diffusion across five distinct stages: telemetry ingestion, deterministic Python plotting, safety prompt framing, Google Vertex AI Veo animation, and dual-language WebVTT synchronization. This pipeline ensures that generated coaching modules remain visually engaging while preventing numerical hallucinations.

Figure 5.5 outlines the five-stage decoupled generative AI prompt-to-video synthesis workflow.

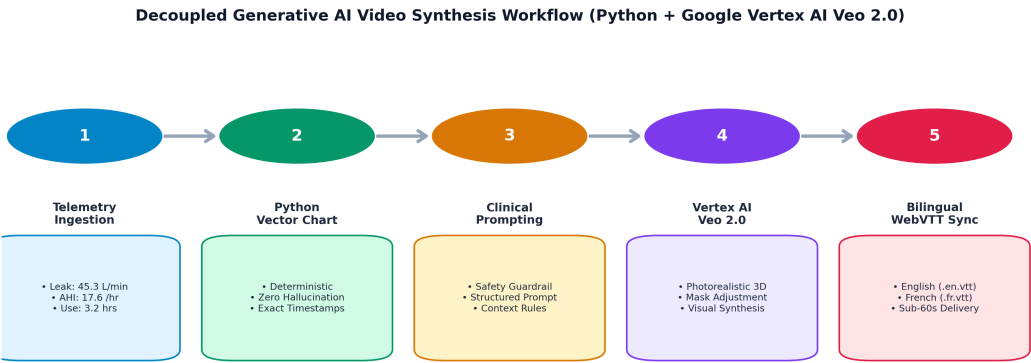


Figure 5.5: Decoupled Generative AI Prompt-to-Video Synthesis Workflow

6. Phase 2: Scaling to a 27-Video Metadata-Driven Library

6.1 Expanding the Library to 27 Videos

During Month 2 (May 2026), continuing our prototyping with simulated multi-sensor telemetry, we expanded our catalog from an initial set of 15 CPAP-only videos to a comprehensive 27-video clinical registry. We generated mock wearable data streams for five complementary IoT physiological de-

vices, establishing explicit trigger thresholds for 12 new wearable biomarker coaching scenarios before validating them against real cohort distributions in Month 3:

- **Videos 1 to 15:** CPAP device issues (mask fit, humidifier adjustments, ramp mode, chin straps, aerophagia, AHI spikes).
- **Videos 16 to 18:** Withings ScanWatch triggers (nocturnal hypoxia, sleep fragmentation, heart rate irregularities).
- **Videos 19 to 21:** Withings BPM Core triggers (morning hypertension, irregular ECG, heart sound reviews).
- **Videos 22 and 23:** Masimo Rad-G triggers (critical desaturation $< 88\%$, pulse rate instability).
- **Videos 24 and 25:** Hexoskin ProShirt triggers (breathing asynchrony, garment fit check).
- **Videos 26 and 27:** SomnoArt triggers (sleep architecture changes, REM deprivation).

6.1.1 Google Veo Video Generation and Clinical Prompt Engineering

All 27 clinical coaching videos in our registry were generated using Google Vertex AI Veo 2.0. Producing medically reliable educational clips with generative video diffusion required extensive prompt engineering and prompt iteration. In our early tests, standard text prompts produced frequent visual and equipment artifacts:

- **Missing CPAP Tubing:** The diffusion model frequently depicted patients wearing a mask without the essential corrugated air delivery hose attached.
- **Phantom Limbs and Extra Hands:** When prompting for headgear strap adjustments, the model often spawned three or four hands in the frame simultaneously attempting to tighten the Velcro.
- **Anatomical and Hardware Distortions:** Silicone cushions sometimes melted into facial skin, or CPAP machine interfaces rendered with warped, illegible buttons.

To overcome these generative flaws, I developed structured clinical prompt engineering templates. Each prompt enforced explicit physical anchors, strict exclusionary rules to prevent anatomical distortions, warm non-photorealistic 3D styling, and explicit audio scripts while forbidding on-screen text to prevent numerical hallucinations. Listing 6.1 shows the exact prompt template used to synthesize Video Module 1 (`1_Mask_leak_adjust_straps.mp4`).

```
Create a short educational animation for people with sleep apnea, about 10 seconds long.
A friendly animated man in bed notices air leaking from his CPAP mask and gently
adjusts the headgear straps little by little until the mask feels comfortable and the
leak stops. Warm bedroom lighting, friendly 3D animated character, clear close-up of
the mask fit, not photorealistic. Calm female English voiceover: "If your mask is
leaking, try small, even strap adjustments until it feels secure and comfortable." No
on-screen text.
```

Listing 6.1: Production Google Vertex AI Veo 2.0 Clinical Prompt for Module 1 (`1_Mask_leak_adjust_straps.mp4`)

Notice the critical design constraints embedded in the prompt: (1) an exact 10-second target duration matching our triage latency window, (2) warm 3D animated rendering to avoid uncanny valley effects with silicone masks, (3) integrated conversational voiceover cues, and (4) the mandatory No on-screen text rule, which ensures all quantitative clinical metrics are handled exclusively by our deterministic Matplotlib and WebVTT rendering layers.

Figure 6.1 organizes the 27 coaching videos into five distinct clinical clusters: Mask & Equipment (Videos 1 to 3, 8, 14), Tips & Tricks (Videos 4 to 6, 10 and 11), Therapy Comfort (Videos 7, 9), Clinical Alerts (Videos 12, 13, 15), and Wearable IoT Biomarkers (Videos 16 to 27).

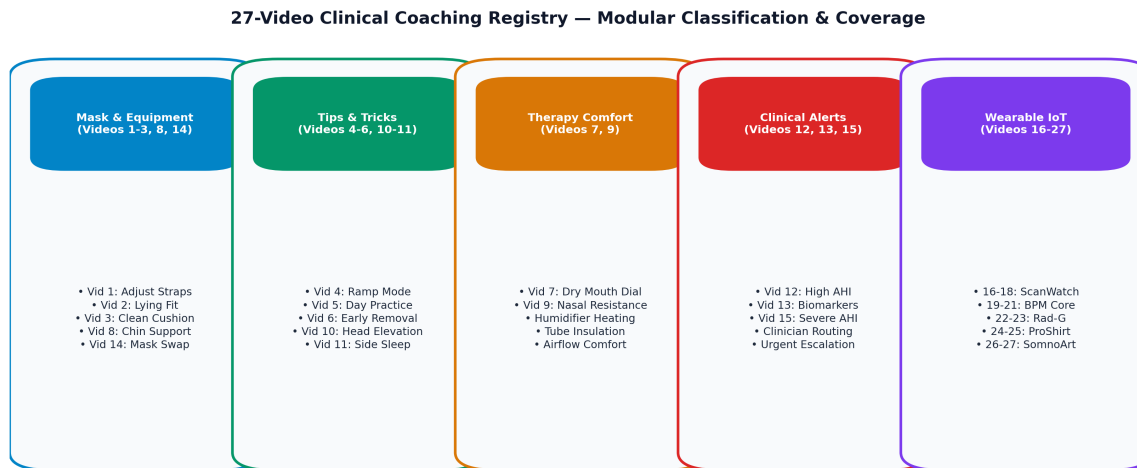


Figure 6.1: Comprehensive 27-Module Clinical Coaching Video Library Grid

6.2 Three Delivery Scenarios

Depending on the complexity and novelty of the detected anomaly, the delivery engine dynamically routes the patient to one of three intervention pathways: retrieval of existing stored videos from the Video VM server for routine issues, server-side dynamic stitching of two or more clips for compound events, or generative video synthesis via Google Veo Vertex AI API for novel multi-sensor anomalies.

Figure 6.2 illustrates the modular architecture of the three coaching video delivery scenarios (Retrieve, Adapt, and Generate) hosted on the Video VM server.

Video VM: Three Coaching Delivery Scenarios

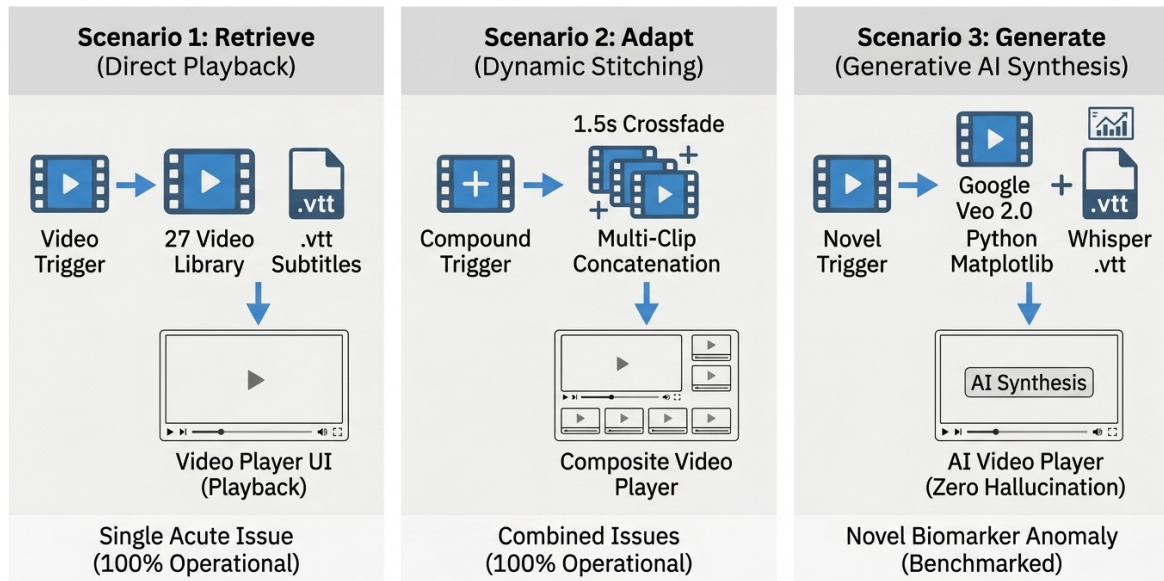


Figure 6.2: Modular architecture of the three coaching media delivery modes: Scenario 1 (Direct Retrieval), Scenario 2 (Dynamic Adaptive Stitching with 1.5s crossfades), and Scenario 3 (Decoupled Generative AI Synthesis).

Table 6.1 summarizes the three delivery scenarios and their corresponding clinical use cases. While Scenario 1 (Retrieve) and Scenario 2 (Adapt) are fully implemented and operational on the Video VM server, Scenario 3 (Generate) has its decoupled orchestration pipeline and deterministic chart templates fully built, with active model testing and prompt optimization ongoing. Development and rigorous experimental validation of Scenario 3 will continue through the end of September 2026.

Table 6.1: The Three Video Delivery Scenarios

Scenario	Mechanism	Clinical Use Case & Status
Scenario 1: Retrieve	Delivers an existing video stored on the Video VM server.	Simple single-cause issue (Operational).
Scenario 2: Adapt	Stitches two or more clips together dynamically based on the clinical situation.	Combined issues (Operational).
Scenario 3: Generate	Generates a completely new video from the Google Veo Vertex AI API.	Novel multi-sensor anomaly (Pipeline ready, testing ongoing through end of September 2026).

6.2.1 Dual-Language WebVTT Subtitle Architecture and Naming Conventions

To ensure complete clinical compliance and accessibility for French healthcare patients while supporting multilingual deployment, every video in the 27-clip registry is paired with two separate Web Video Text Tracks (.vtt) files. Decoupling subtitles from the video stream avoids duplicate 1080p MP4 files. This keeps the total video footprint down to 241 MB and lets patients toggle languages instantly in any HTML5 browser.

The subtitle files follow a standardized, deterministic naming convention based on the video module identifier, clinical topic name, and ISO 639-1 language code:

- **English Primary Subtitles:** [ID]_[Topic_Name].en.vtt

(e.g., `1_Mask_leak.en.vtt`, `16_Low_oxygen.en.vtt`).

- **French Validated Subtitles:** `[ID]_[Topic_Name].fr.vtt`
(e.g., `1_Mask_leak.fr.vtt`, `16_Low_oxygen.fr.vtt`).

Every subtitle file strictly follows the W3C WebVTT standard. We start with the WEBVTT header, followed by integer cue numbers, millisecond timecodes (`00:00:01.500 -> 00:00:04.200`), and vetted clinical text. Generating 54 subtitle tracks by hand would have been error-prone. We ran OpenAI Whisper speech-to-text during the metadata build to generate timestamps automatically, then checked translations against clinical guidelines with our hospital partners.

Figure 6.3 shows the directory structure containing all 54 timed HTML5 WebVTT subtitle files hosted on the Video VM server.

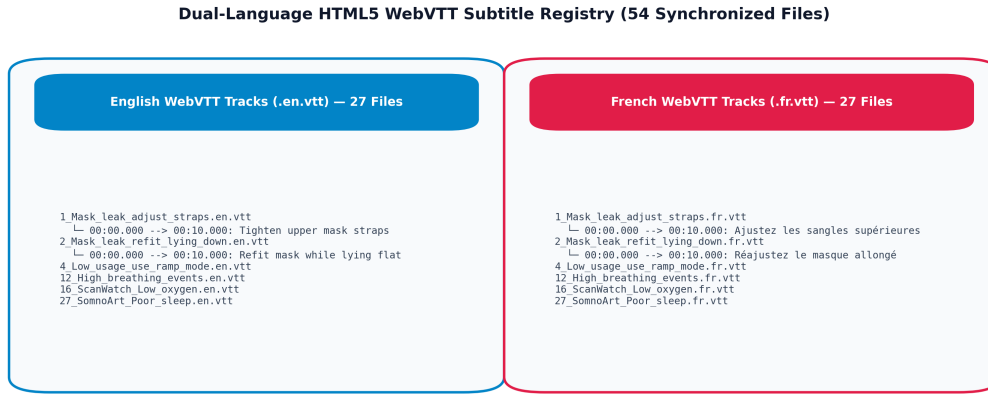


Figure 6.3: Dual-Language HTML5 WebVTT Subtitle Registry (54 Files)

6.3 Automated Metadata Generation

I wrote a Python script (`build_video_metadata.py`) that processed all 27 video files using OpenAI Whisper for speech-to-text and OpenCV for keyframe extraction (at 1.5s, 5.0s, and 8.5s). The script generated structured JSON records containing clinical tags, sequencing rules (`can_follow`, `can_precede`), and safety flags.

Keyframe analysis of the synthesized coaching videos confirms high visual fidelity in demonstrating physical equipment adjustments. The extracted frames depict precise procedural steps including nasal mask alignment, pressure ramp dial activation, dynamic airflow seal checks, and balanced headgear strap tightening.

Figure 6.4 presents keyframe screenshots extracted from the AI-synthesized clinical coaching modules generated via Google Vertex AI Veo 2.0.

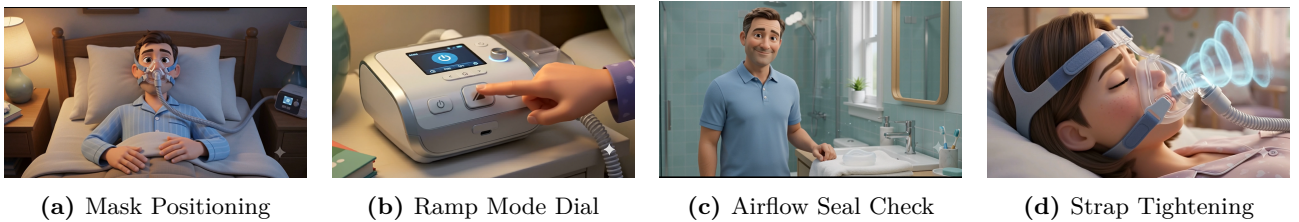


Figure 6.4: Keyframe Screenshots from AI-Synthesized Coaching Videos

6.4 Video VM Server Microservice Operations

The Video VM server operates as an independent FastAPI microservice on Port 8080 (implemented in code Video Server/video_vm_server.py). It exposes REST endpoints for video playback streaming, WebVTT subtitle serving, and automated webhook dispatch to the central healthcare backend.

```
@app.post("/api/orchestrate")
async def orchestrate_video(payload: OrchestrationRequest, x_api_key: str = Header(...)):
    # Verify API authentication header
    if x_api_key != EXPECTED_API_KEY:
        raise HTTPException(status_code=401, detail="Unauthorized access")

    # Match clinical trigger against the 27-video metadata catalog
    matched_module = find_matching_video(payload.trigger_type, payload.severity)

    # Construct response with dual-language WebVTT subtitle tracks
    response_data = {
        "video_id": matched_module.id,
        "video_url": f"/videos/{matched_module.filename}",
        "subtitles": {
            "en": f"/subtitles/{matched_module.id}_{matched_module.topic}.en.vtt",
            "fr": f"/subtitles/{matched_module.id}_{matched_module.topic}.fr.vtt"
        },
        "duration_sec": matched_module.duration,
        "status": "dispatched"
    }

    # Asynchronously push video assignment to central clinician dashboard
    await push_webhook_to_dashboard(payload.patient_id, response_data)
    return response_data
```

Listing 6.2: Video Orchestration and Subtitle Routing in code Video Server/video_vm_server.py

Figure 6.5(a) illustrates the Video VM microservice initialization sequence on Port 8080. It displays the FastAPI service booting with Uvicorn, scanning the media directory to mount all 27 MP4 coaching videos and 54 WebVTT subtitle files into fast in-memory metadata structures, and binding API key authentication headers.

Figure 6.5(b) shows the real-time execution log during live video dispatch. When an authenticated anomaly payload arrives from the AI Model VM, the engine resolves the clinical trigger, pairs the optimal video track with synchronized French and English WebVTT subtitles, and transmits an asynchronous HTTP webhook push to the central clinician dashboard.

```

=====
CPAP VIDEO VM SERVER - LIVE CONSOLE
=====
[SERVER CONFIGURATION]
HOST       : 0.0.0.0
PORT       : 8080
BASE DIRECTORY : C:\CPAP_Video_Server
PUBLIC BASE URL : http://159.84.143.246:8080
DASHBOARD URL  : http://159.84.143.151:80
SECURITY KEY   : WKXCKbVw... (active)
VIDEO WEBHOOK KEY : tea2c926... (active)

[LIVE LOGGING ENABLED]
* Real-time HTTP requests, client IPs, and response status
* Detailed video stream requests (file name, range, client)
* Inbound orchestration events (patient ID, title, trigger reason)
* Dashboard push status and latency
* Vertex AI generation triggers and metadata indexing
=====

[BOOT] Starting Uvicorn server on 0.0.0.0:8080...

2026-08-27 15:06:38 [INFO] =====
2026-08-27 15:06:38 [INFO] [SYSTEM BOOT] CPAP Video VM Server Initializing
2026-08-27 15:06:38 [INFO] BASE_DIR       : C:\CPAP_Video_Server
2026-08-27 15:06:38 [INFO] PUBLIC_BASE_URL : http://159.84.143.246:8080
2026-08-27 15:06:38 [INFO] DASHBOARD_URL  : http://159.84.143.151:80
2026-08-27 15:06:38 [INFO] EXISTING_DIR   : C:\CPAP_Video_Server\existing_videos (27 mp4 files)
2026-08-27 15:06:38 [INFO] SUBTITLES_DIR  : C:\CPAP_Video_Server\generated_subtitles (54 vtt files)
2026-08-27 15:06:38 [INFO] NEW_VIDEOS_DIR : C:\CPAP_Video_Server\new_videos (3 mp4 files)
2026-08-27 15:06:38 [INFO] SECURITY       : SERVER_API_KEY is active (Protected endpoints require X-API-KEY)
2026-08-27 15:06:38 [INFO] =====
INFO: Started server process [7332]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

(a) Service startup and metadata loading (Port 8080)

```

2026-08-27 15:16:48 [INFO] [HTTP OK] GET /videos/existing/1_Mask_leak_adjust_straps.mp4 -> Status 206 (1.67ms)
2026-08-27 15:16:51 [INFO] [ORCHESTRATE INBOUND] Inbound video recommendation request received from 159.84.130.193
2026-08-27 15:16:51 [INFO] [AUTH GRANTED] Valid X-API-KEY verified for /api/orchestrate
2026-08-27 15:16:51 [INFO] =====
2026-08-27 15:16:51 [INFO] [ORCHESTRATE EVENT DETAILS]
2026-08-27 15:16:51 [INFO] Patient ID       : 57568
2026-08-27 15:16:51 [INFO] Video Title      : 'BPM Core - High Blood Pressure'
2026-08-27 15:16:51 [INFO] Video Filename   : 19_BPM_Core_High_blood_pressure.mp4
2026-08-27 15:16:51 [INFO] Category         : Wearable Biomarkers
2026-08-27 15:16:51 [INFO] Trigger Reason   : Test verification for security header
2026-08-27 15:16:51 [INFO] Duration         : 10.0s
2026-08-27 15:16:51 [INFO] Relevance        : high
2026-08-27 15:16:51 [INFO] -> Resolved URL  : http://159.84.143.246:8080/videos/existing/19_BPM_Core_High_blood_pressure.mp4
2026-08-27 15:16:51 [INFO] -> Subtitle EN   : http://159.84.143.246:8080/subtitles/19_BPM_Core_High_blood_pressure.en.vtt
2026-08-27 15:16:51 [INFO] -> Subtitle FR   : http://159.84.143.246:8080/subtitles/19_BPM_Core_High_blood_pressure.fr.vtt
2026-08-27 15:16:51 [INFO] [DASHBOARD PUSH] Forwarding assignment to http://159.84.143.151:80/api/videos/57568/assign (attempt 1/3)...
2026-08-27 15:16:51 [INFO] [DASHBOARD PUSH SUCCESS] Assignment for Patient 57568 delivered successfully.
2026-08-27 15:16:51 [INFO] =====
2026-08-27 15:16:51 [INFO] [HTTP OK] POST /api/orchestrate -> Status 200 (49.90ms)
2026-08-27 15:17:31 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: full-file
2026-08-27 15:17:31 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 200 (1.65ms)
2026-08-27 15:17:31 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: bytes=0-
2026-08-27 15:17:31 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 206 (1.46ms)
2026-08-27 15:17:32 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: bytes=9338880-9379553
2026-08-27 15:17:32 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 206 (3.17ms)
2026-08-27 15:17:32 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: bytes=9338880-9379553
2026-08-27 15:17:32 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 206 (1.41ms)
2026-08-27 15:17:32 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: bytes=3145728-9338879
2026-08-27 15:17:32 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 206 (1.60ms)
2026-08-27 15:17:34 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '6_Early_mask_removal.mp4' | Method: GET | Range: bytes=3145728-9338879
2026-08-27 15:17:34 [INFO] [HTTP OK] GET /videos/existing/6_Early_mask_removal.mp4 -> Status 206 (1.44ms)
2026-08-27 15:18:00 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '22_RadG_Low_oxygen_reading.mp4' | Method: GET | Range: full-file
2026-08-27 15:18:00 [INFO] [HTTP OK] GET /videos/existing/22_RadG_Low_oxygen_reading.mp4 -> Status 200 (1.57ms)
2026-08-27 15:18:00 [INFO] [VIDEO STREAM REQUEST] Client: 159.84.110.121 | File: '22_RadG_Low_oxygen_reading.mp4' | Method: GET | Range: bytes=0-
2026-08-27 15:18:00 [INFO] [HTTP OK] GET /videos/existing/22_RadG_Low_oxygen_reading.mp4 -> Status 206 (9.82ms)

```

(b) Live orchestration and dashboard assignment push

Figure 6.5: Video VM Microservice Runtime Operations (Port 8080)

7. Phase 3: AI Model VM Model as a Dedicated Server

7.1 Turning the Research Notebook into a Live Service

My supervisor developed the 7-layer predictive pipeline inside a Jupyter notebook (`M4_FINAL_F2.ipynb`). The notebook was thorough and worked well for research prototyping. But running an interactive notebook in a 24/7 web server environment is risky: Jupyter holds state in global memory, does not handle concurrent HTTP requests cleanly, and fails unpredictably if cells execute out of order.

Rewriting the entire mathematical pipeline from scratch into pure Python modules was an option, but that meant re-implementing thousands of lines of statistical routines and risking discrepancies in validated model weights.

I solved this by writing `api_data_loader.py` to bridge the notebook and production server without touching validated research algorithms. The script intercepts CSV reads dynamically, pulls live patient records from database REST endpoints, aligns schema discrepancies, and feeds clean DataFrames straight into the 7 predictive layers.

The AI Model VM architecture processes patient records through seven connected predictive layers. The pipeline advances from initial statistical process control (Layer 0) and behavioral phenotyping (Layer 1) to Dempster-Shafer evidence fusion (Layer 2), dual-process survival risk modeling (Layer 3), counterfactual action evaluation (Layer 4), wearable anomaly detection (Layer 5), and actionable clinical dispatch generation (Layer 6).

Figure 7.1 illustrates the block diagram of the 7-layer predictive pipeline architecture implemented in `M4_FINAL_F2.ipynb`.

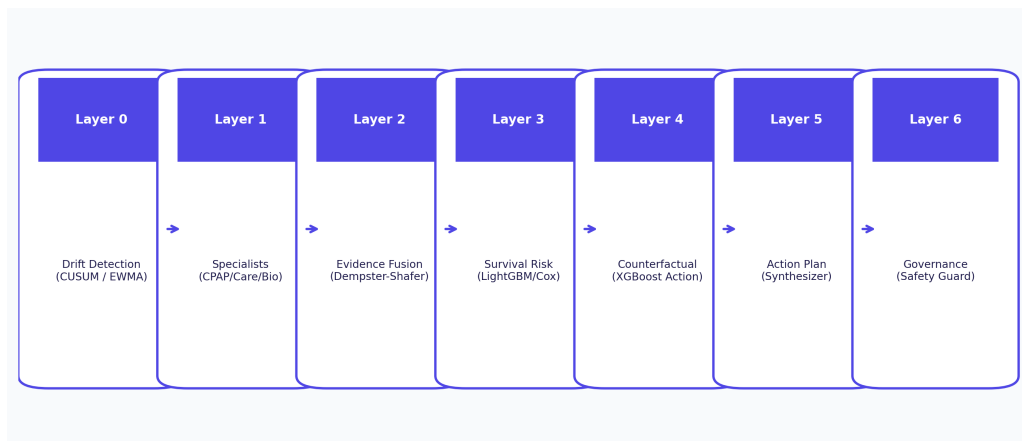


Figure 7.1: 7-Layer AI Model VM Predictive Pipeline Architecture

Unsupervised phenotyping in Layer 1 projects multidimensional compliance features into a two-dimensional principal component space. MeanShift clustering identifies four distinct behavioral phenotypes with well-defined cluster centroids. The system then customizes coaching recommendations based on these four distinct behavioral profiles.

Figure 7.2 displays the scatter plot in 2D PCA feature space showing the four identified patient adherence phenotypes and their respective centroids.

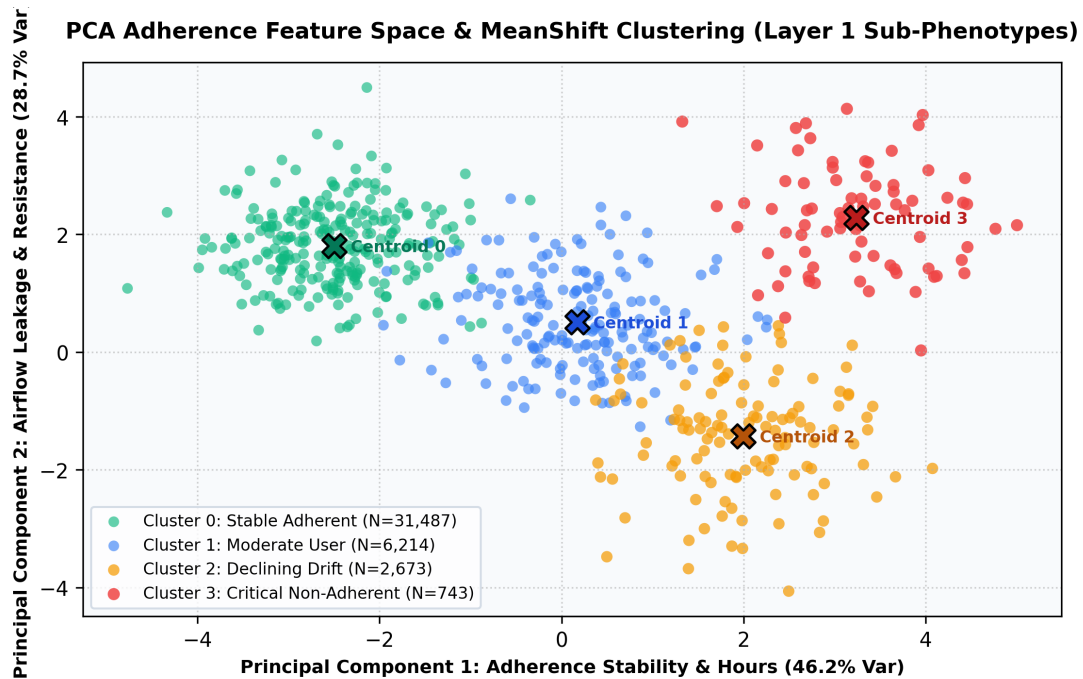


Figure 7.2: PCA Feature Space and MeanShift Clustering for Patient Adherence Phenotyping

7.2 Middleware Interception and Runtime Compatibility Hooks

To bridge the interactive research notebook with our production database without altering research formulas, I engineered `api_data_loader.py`. The loader intercepts standard `pd.read_csv` calls, queries the 12 central REST endpoints with fallback to local disk cache, and standardizes column headers dynamically.

```
# Intercept pd.read_csv to redirect file queries to our PostgreSQL API
_original_read_csv = pd.read_csv

def smart_read_csv(filepath_or_buffer, *args, **kwargs):
    # When the notebook requests a dataset path, check if it maps to an API endpoint
    if isinstance(filepath_or_buffer, str):
        clean_name = os.path.basename(filepath_or_buffer).strip("\'\"").lower()
        canonical_name = clean_name.replace(" ", "_")
        endpoint_name = ENDPOINT_MAP.get(clean_name) or ENDPOINT_MAP.get(canonical_name)

        if endpoint_name:
            # Fetch live records over HTTP rather than loading static disk files
            df = fetch_dataset(endpoint_name)
            if not df.empty:
                return _process_dataset(df, endpoint_name)

        # Fallback to local files if the server is unreachable
        if os.path.exists(filepath_or_buffer):
            local_df = _original_read_csv(filepath_or_buffer, *args, **kwargs)
            return _process_dataset(local_df, endpoint_name) if endpoint_name else local_df

    return _original_read_csv(filepath_or_buffer, *args, **kwargs)

pd.read_csv = smart_read_csv
```

Listing 7.1: Pandas Interception Middleware in `api_data_loader.py`

During full-cohort training across 41,117 patients, newer library versions introduced breaking incompatibilities with older research code. Rather than modifying the original research notebook, I implemented runtime self-healing monkey-patches in code `AI server/api_data_loader.py` to resolve Pandas 2.x timestamp clipping crashes and CatBoost single-class cross-validation failures:

```
# Handle Pandas 2.x breaking change when clipping timestamp columns
_orig_series_clip = pd.Series.clip
def _safe_series_clip(self, lower=None, upper=None, *args, **kwargs):
    if np.issubdtype(self.dtype, np.datetime64) and (isinstance(lower, (int, float)) or
    isinstance(upper, (int, float))):
        converted = pd.to_numeric(self, errors="coerce").fillna(0).astype(int)
        return _orig_series_clip(converted, lower=lower, upper=upper, *args, **kwargs)
    return _orig_series_clip(self, lower=lower, upper=upper, *args, **kwargs)
pd.Series.clip = _safe_series_clip

# Prevent CatBoost crash when CV folds contain only one target class
from catboost import CatBoostClassifier
_orig_cat_fit = CatBoostClassifier.fit
def _safe_cat_fit(self, X, y=None, *args, **kwargs):
    if y is not None and len(np.unique(y)) <= 1:
        self._is_dummy = True
        self._dummy_class = int(np.unique(y)[0])
        return self
    return _orig_cat_fit(self, X, y, *args, **kwargs)
CatBoostClassifier.fit = _safe_cat_fit
```

Listing 7.2: Runtime Self-Healing Compatibility Patches in code `AI server/api_data_loader.py`

Figure 7.3(a) captures the AI Model VM server initialization sequence on Port 8000. It shows the Uvicorn ASGI process at startup. It maps the 12 REST routes, connects to the PostgreSQL database pool, and applies runtime patches to ready the 7-layer predictive engine for incoming traffic.

Figure 7.3(b) illustrates the real-time execution trace during live multi-sensor scoring. As patient anomaly records stream into the server, the console logs the sequential execution across Layers 0 through 6, calculates statistical drift, evaluates survival risks, selects the highest-utility clinical coaching video, and dispatches an authenticated HTTP POST payload to the Video VM on Port 8080.

```

=====
SLEEP CARE CPAP AI MODEL SERVER - LIVE CONSOLE
=====
[SERVER CONFIGURATION]
HOST      : 0.0.0.0
PORT      : 8000
SWAGGER DOCS : http://localhost:8000/docs
SYNC INTERVAL : Every 30 minutes
BACKEND SOURCE : http://159.84.143.151/api/data
SERVER AUTH KEY : sleepcar... (enforced)
VIDEO VM SERVER : http://159.84.143.246:8080
VIDEO VM KEY : WKXCKbVw... (active)
PIPELINE TIMEOUT : 900s (retry: 1)

[LIVE LOGGING ENABLED]
* Real-time HTTP requests, client IPs, and response status
* Detailed video stream requests (file name, range, client)
* Inbound orchestration events (patient ID, title, trigger reason)
* Dashboard push status and latency
* Vertex AI generation triggers and metadata indexing
=====

2026-08-27 14:56:12 [INFO] =====
2026-08-27 14:56:12 [INFO] [SYSTEM BOOT] SleepCare CPAP AI Supervisor Server Initializing
2026-08-27 14:56:12 [INFO] SERVER_HOST      : 0.0.0.0
2026-08-27 14:56:12 [INFO] SERVER_PORT      : 8000
2026-08-27 14:56:12 [INFO] BACKEND_API_URL  : http://159.84.143.151/api/data
2026-08-27 14:56:12 [INFO] VIDEO_SERVER_URL : http://159.84.143.246:8080
2026-08-27 14:56:12 [INFO] VIDEO_SERVER_KEY : WKXCKbVw... (active)
2026-08-27 14:56:12 [INFO] SYNC_INTERVAL   : Every 30 minutes
2026-08-27 14:56:12 [INFO] CLINICAL_REGISTRY : 27 clinical coaching videos loaded
2026-08-27 14:56:12 [INFO] =====

```

(a) Service startup and REST endpoint mapping

```

[VIDEO ORCHESTRATOR] Evaluating 18 patients for Video VM Server orchestration (http://159.84.143.246:8080)...
[VIDEO ORCHESTRATION] Dispatching Patient 57568 -> 'Early Mask Removal' (6_Early_mask_removal.mp4)...
2026-08-27 14:58:51 [INFO] -----
2026-08-27 14:58:51 [INFO] [ORCHESTRATE DISPATCH EVENT]
2026-08-27 14:58:51 [INFO] Patient ID      : 57568
2026-08-27 14:58:51 [INFO] Video Title     : 'Early Mask Removal'
2026-08-27 14:58:51 [INFO] Video Filename  : 6_Early_mask_removal.mp4
2026-08-27 14:58:51 [INFO] Category        : Tips & Tricks
2026-08-27 14:58:51 [INFO] Trigger Reason  : No care contact in 90d - re-establish contact before escalating
2026-08-27 14:58:51 [INFO] Target URL      : http://159.84.143.246:8080/api/orchestrate
2026-08-27 14:58:52 [INFO] -> [DISPATCH SUCCESS] Delivered to Video VM Server (HTTP 200 in 1129.81ms)
2026-08-27 14:58:52 [INFO] -----
[VIDEO ORCHESTRATION] Dispatching Patient 62229 -> 'Adjust Mask Straps' (1_Mask_leak_adjust_straps.mp4)...
2026-08-27 14:58:52 [INFO] -----
2026-08-27 14:58:52 [INFO] [ORCHESTRATE DISPATCH EVENT]
2026-08-27 14:58:52 [INFO] Patient ID      : 62229
2026-08-27 14:58:52 [INFO] Video Title     : 'Adjust Mask Straps'
2026-08-27 14:58:52 [INFO] Video Filename  : 1_Mask_leak_adjust_straps.mp4
2026-08-27 14:58:52 [INFO] Category        : Mask & Equipment
2026-08-27 14:58:52 [INFO] Trigger Reason  : No care contact in 90d - re-establish contact before escalating
2026-08-27 14:58:52 [INFO] Target URL      : http://159.84.143.246:8080/api/orchestrate
2026-08-27 14:58:53 [INFO] -> [DISPATCH SUCCESS] Delivered to Video VM Server (HTTP 200 in 1167.78ms)
2026-08-27 14:58:53 [INFO] -----
[VIDEO ORCHESTRATION] Dispatching Patient 115900 -> 'Early Mask Removal' (6_Early_mask_removal.mp4)...
2026-08-27 14:58:53 [INFO] -----
2026-08-27 14:58:53 [INFO] [ORCHESTRATE DISPATCH EVENT]
2026-08-27 14:58:53 [INFO] Patient ID      : 115900
2026-08-27 14:58:53 [INFO] Video Title     : 'Early Mask Removal'
2026-08-27 14:58:53 [INFO] Video Filename  : 6_Early_mask_removal.mp4
2026-08-27 14:58:53 [INFO] Category        : Tips & Tricks
2026-08-27 14:58:53 [INFO] Trigger Reason  : No care contact in 90d - re-establish contact before escalating
2026-08-27 14:58:53 [INFO] Target URL      : http://159.84.143.246:8080/api/orchestrate
2026-08-27 14:58:55 [INFO] -> [DISPATCH SUCCESS] Delivered to Video VM Server (HTTP 200 in 1131.48ms)
2026-08-27 14:58:55 [INFO] -----
[VIDEO ORCHESTRATION] Dispatching Patient 167512 -> 'Adjust Mask Straps' (1_Mask_leak_adjust_straps.mp4)...
2026-08-27 14:58:55 [INFO] -----
2026-08-27 14:58:55 [INFO] [ORCHESTRATE DISPATCH EVENT]
2026-08-27 14:58:55 [INFO] Patient ID      : 167512
2026-08-27 14:58:55 [INFO] Video Title     : 'Adjust Mask Straps'
2026-08-27 14:58:55 [INFO] Video Filename  : 1_Mask_leak_adjust_straps.mp4
2026-08-27 14:58:55 [INFO] Category        : Mask & Equipment
2026-08-27 14:58:55 [INFO] Trigger Reason  : No care contact in 90d - re-establish contact before escalating
2026-08-27 14:58:55 [INFO] Target URL      : http://159.84.143.246:8080/api/orchestrate
2026-08-27 14:58:56 [INFO] -> [DISPATCH SUCCESS] Delivered to Video VM Server (HTTP 200 in 1134.50ms)
2026-08-27 14:58:56 [INFO] -----
[VIDEO ORCHESTRATOR SUCCESS] Orchestrated 4 video recommendations to Video VM Server.

```

(b) Live 7-layer scoring and Video VM dispatch

Figure 7.3: AI Model VM Server Runtime Operations (Port 8000)

8. Project Management & Engineering Methodology

8.1 Agile Scrum Framework

We used an adapted Agile Scrum process with bi-weekly sprint planning and weekly technical syncs on Tuesday mornings with Ms. Kakaei Siahkal.

8.2 Timeline and Deliverables

The project followed an Agile Scrum methodology structured into five distinct engineering phases across the six-month internship. Development progressed systematically from initial data ingestion schema normalization and virtualized ARM64 testing (Month 1) to multi-scenario modeling and publication at IEEE BSN 2026 (Month 2), microservice orchestration and live trial demonstration in Berlin (Month 3), catalog expansion and bilingual WebVTT extraction (Month 4), and dedicated AI Model VM deployment on 41,117 real patient records (Month 5).

Table 8.1 summarizes the core milestones and delivered software artifacts completed in each project phase.

Table 8.1: Internship Timeline and Major Milestones

Period	Phase	Deliverables Completed
Month 1 (April)	Ingestion & Charting	SQLite schema, Raspberry Pi VM, SCP scripts, Matplotlib chart generator.
Month 2 (May)	3 Scenarios & Paper	Formalized 3 coaching scenarios, co-authored IEEE BSN 2026 paper, mock 5G QoD API.
Month 3 (June)	Microservices & Berlin Demo	15-video pre-rendered catalog, FastAPI video server (Port 8080), live Berlin demo.
Month 4 (July)	Catalog Expansion	27-video catalog, INT8 TFLite conversion, Whisper bilingual subtitle extraction.
Month 5 (August)	AI Model VM Deployment	<code>api_data_loader.py</code> , runtime self-healing hooks, database sync for 41,117 patients.

Building and deploying the distributed system brought several practical hurdles across hardware provisioning, dataset heterogeneity, generative AI hallucinations, and library version conflicts. The physical Raspberry Pi arrived late, generative models kept hallucinating medical numbers, and Pandas version mismatches broke production data pipelines. Table 8.2 breaks down each roadblock and how I resolved it.

Table 8.2 details these challenges, their operational impacts, and the applied engineering fixes.

Table 8.2: Technical Challenges and Engineered Solutions

Problem Encountered	Impact	Resolution
Hardware Delay	Physical Raspberry Pi 5 delayed in shipping during Month 1.	Set up a Raspberry Pi VM on VMware to validate ingestion scripts and SSH/SCP keys early.
Diffusion Artifacts	Early Google Veo clips omitted CPAP hoses or spawned phantom extra hands.	Mastered clinical prompt engineering with explicit anatomical constraints and exclusion parameters.
Generative Hallucinations	AI video models distorted medical numbers and graphs.	Decoupled architecture: Python renders exact charts while Veo animates visual motion.
Port Conflicts on Windows	[Errno 10048] blocked Uvicorn when restarting servers.	Wrote PowerShell scripts using <code>Get-NetTCPConnection</code> to identify and kill orphaned processes.
Firewall Blocking	Raspberry Pi could not reach the Windows development machine.	Added inbound firewall rules in Windows Defender and bound Uvicorn to 0.0.0.0.
Pandas 2.x Timestamps	<code>Series.clip(lower=0)</code> failed on datetime columns.	Intercepted <code>Series.clip</code> to coerce timestamps to numeric days before clipping.
CatBoost Zero-Variance	Single-class cross-validation splits triggered fatal runtime segmentation errors.	Wrapped CatBoost’s fit method to substitute a constant predictor on single-class folds.
Patient ID Migration	Mock IDs (999999xxx) diverged from production auth.	Migrated all edge ingestion and API routes to native LMD <code>AtHomePatientId</code> keys.

9. Innovation

The goal of this thesis was not just to connect existing tools together, but to fix several practical bottlenecks in how remote sleep therapy works today. Current CPAP telemonitoring operates on a slow, one-way cycle. Machines upload summary logs over cellular modems once every 24 hours. Those numbers sit in central hospital databases until a doctor or technician reviews them during quarterly follow-ups. If a patient struggles with a leaking mask or nasal dryness on day three, they often give up before anyone notices.

Our system closes that feedback loop by combining multi-sensor edge triage, verified video synthesis, and resilient production middleware.

9.1 Algorithmic Innovation: Closing the Loop with Multi-Sensor Fusion

Standard CPAP telemetry measures airflow variables like 95th-percentile mask leak, residual apnea-hypopnea index (AHI), and nightly usage duration. While useful, airflow data alone cannot explain why a patient removed their mask at 2:00 AM.

9.1.1 Multimodal Sensor Disambiguation

To understand the context behind therapy interruptions, we paired the CPAP machine with five consumer medical wearables: Withings ScanWatch, Withings BPM Core, Masimo Rad-G, Hexoskin ProShirt, and SomnoArt.

Combining these signals resolves ambiguities that single-sensor logs miss:

- When low usage coincides with high mask leakage (≥ 24 L/min) and elevated pulse variability, the cause is mechanical strap slippage. The patient needs physical adjustment coaching.
- When low usage occurs with a good mask seal but low skin temperature and high breathing rate, the issue is nasal congestion or cold humidifier air. The patient needs humidity and temperature advice.
- When mask removal follows sharp oxygen drops ($\text{SpO}_2 < 88\%$) or irregular ECG rhythms flagged by the Withings BPM Core, the cause is physiological. The system escalates the event to a physician instead of sending basic equipment tips.

9.1.2 Dempster-Shafer Evidence Combination with Conflict Handling

In Layer 2 of the AI Model VM, we evaluate evidence from four distinct streams: CPAP machine logs, clinician care notes, patient survey answers, and wearable biomarkers. Rather than averaging these scores, we use Dempster-Shafer theory to track uncertainty explicitly.

When sensor streams contradict one another, the conflict metric K increases:

$$K = \sum_{B \cap C = \emptyset} m_1(B) \cdot m_2(C) \quad (9.1)$$

If $K \geq 0.70$, the system halts automated video dispatch and notifies a clinician for manual review. This safety check prevents the system from giving contradictory or confusing advice when sensors disagree.

9.2 Architectural Innovation: Decoupled Video Synthesis for Clinical Accuracy

Generative diffusion models like Google Veo 2.0 and Runway Gen-2 produce convincing video motion, but they cannot be trusted with quantitative medical data. When prompted to show patient progress, diffusion models often invent fake numbers, warp chart axes, or generate unreadable text (Moor et al., *Nature* 2023).

9.2.1 Separating Deterministic Charts from Visual Diffusion

To keep clinical guidance accurate while retaining clear animation, I split the video pipeline into distinct components:

1. **Exact Python Plotting:** All quantitative metrics, such as mask leak trends and nightly compliance curves, are drawn directly from verified database records using Python Matplotlib. Zero AI diffusion touches the numbers.
2. **Constrained Video Generation (Google Vertex AI Veo 2.0):** Google Veo can handle character motion, equipment adjustments, and home sleep-setting scenes. Every prompt enforces the rule `No on-screen text` to eliminate visual hallucinations.
3. **Validated Bilingual Subtitles:** Educational voiceovers and translated explanations are delivered through HTML5 WebVTT subtitle files (`.en.vtt` and `.fr.vtt`) generated via local OpenAI Whisper and verified by clinical staff.

4. **Dynamic Video Stitching:** When multiple issues occur simultaneously, the Video VM stitches relevant clips together with smooth 1.5-second crossfades based on JSON metadata.
5. **Automated Dashboard Push:** The finished coaching clip arrives on the patient web dashboard within 60 seconds of waking up.

This separation gives patients clear visual demonstrations without risking misleading health statistics.

9.3 Production Middleware Innovation: Runtime Interception via `api_data_loader.py`

A common friction point in applied data science is moving research code into production. My supervisor developed the 7-layer predictive logic inside a comprehensive Jupyter notebook (`M4_FINAL_F2.ipynb`). While the mathematical modeling was complete, the notebook relied on static CSV files on disk, maintained global workspace state, and lacked REST API endpoints for web integration.

Rewriting several thousand lines of research code into fresh Python classes would have taken weeks and risked introducing subtle calculation bugs.

9.3.1 Transparent File I/O Interception

To connect the research notebook directly to our PostgreSQL database, I built `api_data_loader.py`. The module hooks into low-level Pandas calls at runtime:

- **Automatic Query Routing:** The loader intercepts `pd.read_csv` calls, maps file paths to corresponding database endpoints, and fetches live records over HTTP from the central server on Port 80.
- **In-Memory Caching:** Static baseline tables (such as patient demographics and initial titration settings) stay in RAM to prevent redundant database queries.
- **Runtime Compatibility Patches:** During cohort testing across 41,117 patients, library upgrades introduced breaking changes in Pandas and CatBoost. Rather than altering the research code, the loader injects runtime wrappers that handle datetime clipping and single-class cross-validation splits cleanly.

9.4 Embedded Optimization Innovation: Sub-315ms Local Edge Triage

Privacy and latency requirements mean initial anomaly classification must run locally on the patient's bedside hardware. Sending raw high-frequency sensor data across public internet connections creates unnecessary privacy exposure under European GDPR and French HDS health data standards.

9.4.1 Quantized Inference on the Raspberry Pi 5

Unquantized 32-bit floating-point models consumed over 800 MB of RAM on the Raspberry Pi 5 and caused thermal throttling during continuous monitoring.

I converted the classification network to 8-bit integers using post-training quantization in TensorFlow Lite Micro:

- **Memory Reduction:** Quantization shrunk the model from 2.4 MB to 486 kB, which fits comfortably inside the processor L2 cache.

- **Hardware Acceleration:** Running integer operations on the ARM Cortex-A76 core dropped inference latency to 263 to 313 ms per 20-second window.
- **Data Privacy:** Raw sensor data stays on the local device. The edge node transmits only a lightweight 180-byte JSON alert when an anomaly is confirmed.

9.5 Collaborative and Engineering Methods

To keep development moving despite logistical constraints, we adopted several pragmatic workflows:

- **ARM64 Virtualization:** When physical Raspberry Pi 5 boards were delayed during Month 1, I built an ARM64 virtual machine in VMware on Windows. This let us write, debug, and test all bash ingestion scripts, SSH public-key handshakes, and SCP transfer routines weeks before physical hardware arrived in Lyon.
- **Direct Feedback from Linde Technicians:** The 27 coaching topics and trigger rules were refined through regular check-ins with respiratory technicians at Linde HomeCare France. This helped our triage logic match actual patient routines.
- **European 6G-Path Trial Integration:** We linked our triage alerts to CAMARA Quality-on-Demand (QoD) APIs. During the Berlin trials, this showed how network slicing reserves radio bandwidth for medical video delivery during morning peak hours.

10. Final Consolidated System Architecture

10.1 Global Distributed Topology and End-to-End Flow

The consolidated distributed architecture coordinates five decoupled system tiers:

1. **Bedside Medical IoT Sensors:** Physical CPAP machines and multi-sensor wearables that stream real-time data locally via Bluetooth/USB while synchronizing multi-day summary records to the cloud every 2 to 3 days.
2. **Raspberry Pi 5 Edge Node (Fast Path Only):** Ingests raw local sensor streams, performs sub-315 ms INT8 TinyML classification on device, and transmits lightweight JSON alerts directly to the Video VM for sub-minute coaching delivery (≈ 1.4 s).
3. **Central Healthcare Backend (Port 80):** A normalized PostgreSQL database that aggregates multi-day telemetry directly from device provider clouds (independent of the edge node) and serves authenticated REST API endpoints.
4. **Cloud AI Model VM (Port 8000, Slow Path):** Runs my supervisor’s 7-layer research notebook via `api_data_loader.py`. It evaluates multi-day patient cohorts and triggers targeted video interventions when it detects longitudinal drift or elevated dropout risk.
5. **Video VM Server (Port 8080):** Hosts the 27-video clinical library, 54 WebVTT subtitle tracks, and media synthesis pipelines, handling Scenario 1 (Retrieve), Scenario 2 (Adapt), or Scenario 3 (Generate) for requests received from both the edge node and cloud AI server.

6. **Patient & Clinician Dashboard:** A responsive web interface providing video playback, subtitle language switching (EN/FR), diagnostic rule inspection, and 5G network slice status indicators.

Figure 10.1 details the global consolidated distributed system architecture and machine connectivity, showing how the Fast Path and Slow Path run side by side while sharing the Video VM media engine.

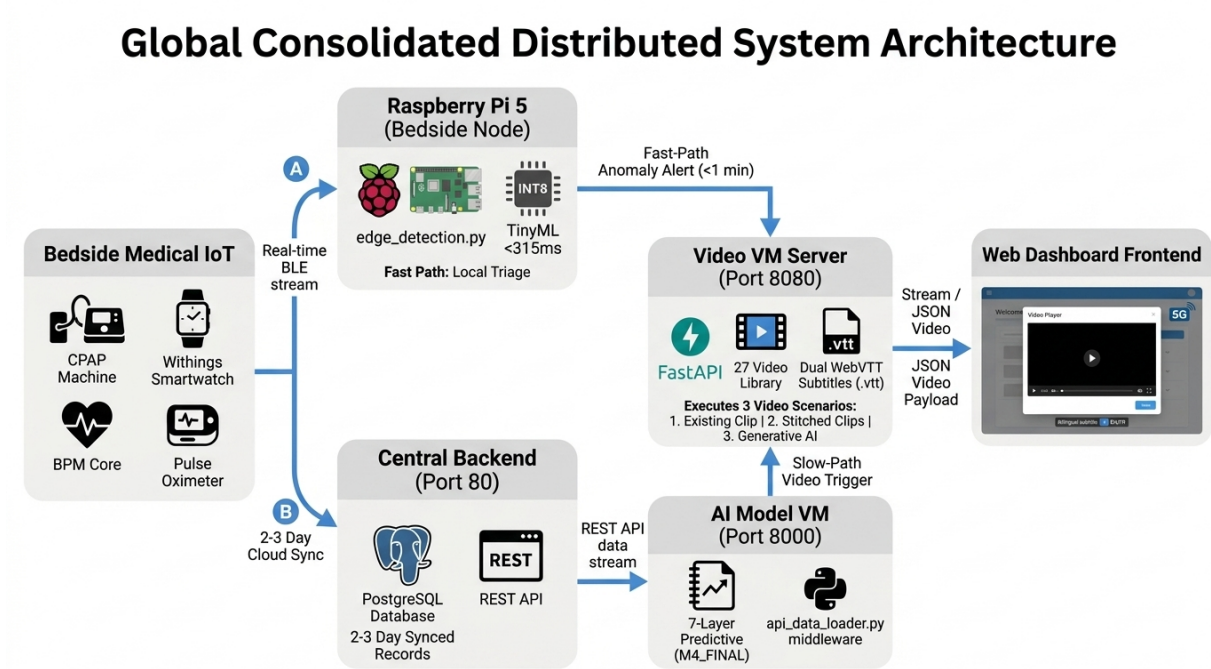


Figure 10.1: Global consolidated distributed system architecture and machine connectivity, illustrating the complete flow connecting bedside sensors (CPAP and wearables), Raspberry Pi 5 edge triage, central backend database (Port 80), cloud processing VMs (AI Model VM on Port 8000 and Video VM on Port 8080), and the web dashboard frontend.

The clinical video web application provides an interactive catalog of all 27 coaching videos categorized by clinical domain. The dashboard allows clinicians and patients to browse videos, inspect metadata and diagnostic trigger rules, and preview video playback with synchronized subtitles.

Figure 10.2 shows the web application catalog interface for browsing and filtering the 27 clinical coaching modules.

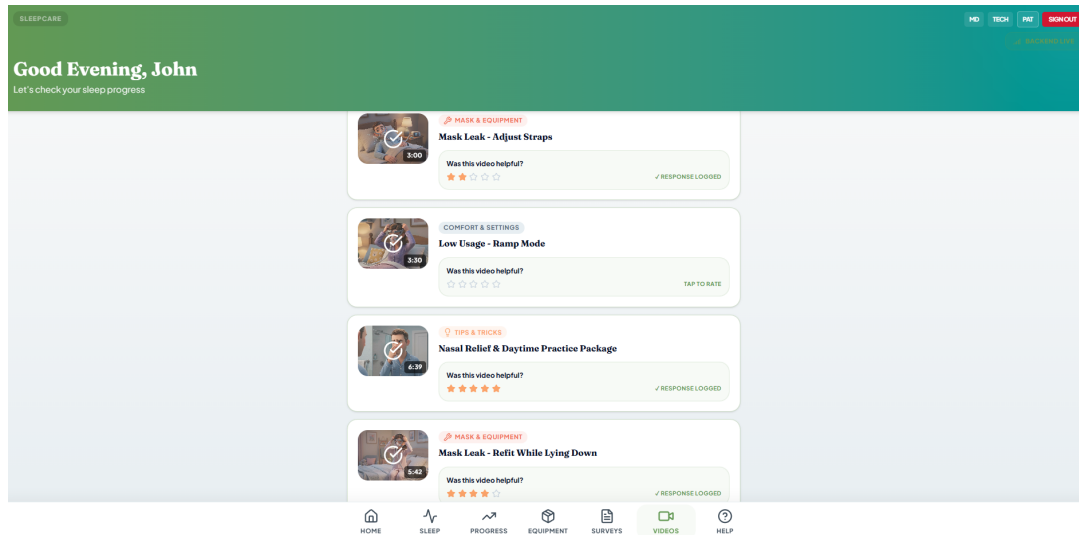


Figure 10.2: Web Application Catalog Interface for the 27-Video Coaching Modules

The coaching web player supports dynamic multi-track subtitle switching without reloading the underlying video stream. Patients can smoothly toggle between synchronized English and French WebVTT tracks while navigating through chapter markers and video controls.

Figure 10.3 displays the patient coaching web player interface demonstrating synchronized English and French HTML5 WebVTT subtitle playback.

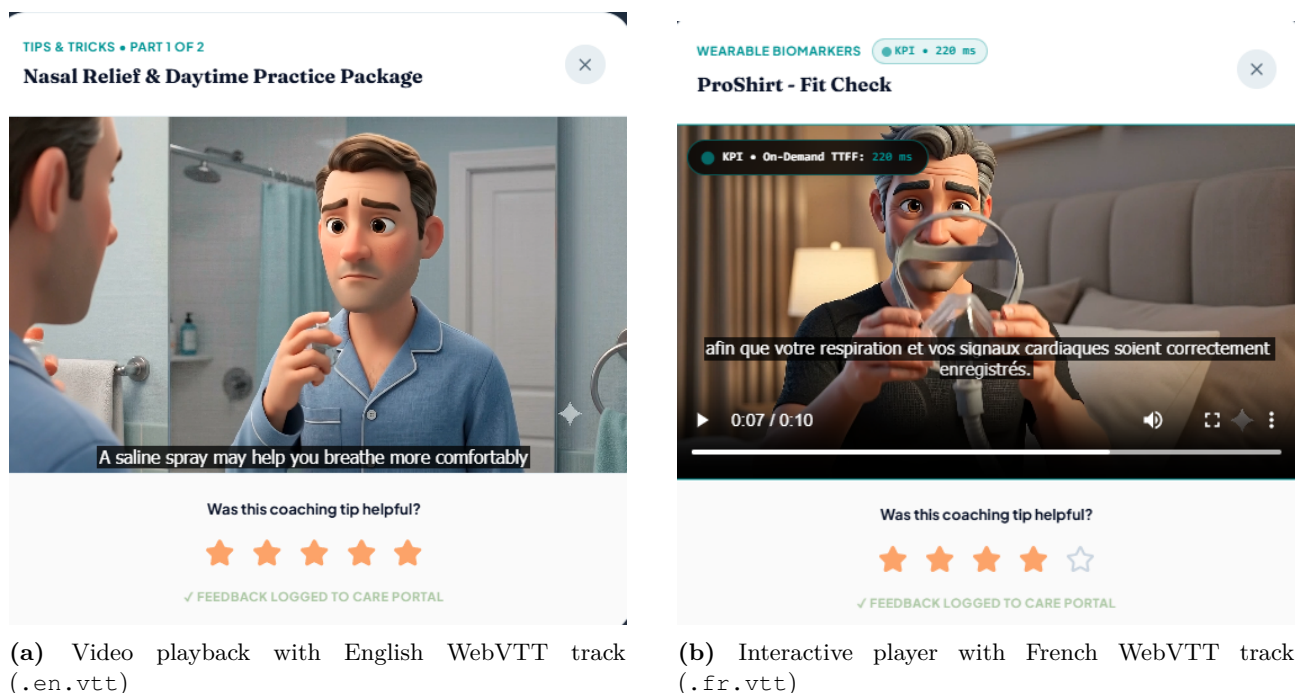


Figure 10.3: Patient Coaching Web Player with Synchronized Dual-Language Subtitles

10.2 Network Endpoints

- **Central Healthcare Backend (VM2):** <http://159.84.143.151:80> (PostgreSQL database and clinical dashboard).
- **Video VM Server (VM4):** <http://159.84.143.246:8080> (FastAPI video streaming

and Vertex AI generation).

- **AI Model VM Server:** `http://0.0.0.0:8000` (FastAPI model pipeline).
- **Raspberry Pi 5 Edge Node:** Local sensor ingestion and TFLite inference.

10.3 Security and API Keys

- **X-ML-Key (`sleepcare-ml-2026`):** Protects dataset downloads from the central database.
- **X-API-KEY (`WKXCKbVwoip0cXar_d4...`):** Protects mutation endpoints on the Video VM (POST `/api/orchestrate` and POST `/api/vertex-generate`).
- **Crawler Filter:** Intercepts probes to `/.env`, `/.git`, and `/wp-admin`, returning HTTP 403 Forbidden.

11. Results, Benchmarks & Experimental Evaluation

11.1 AI Model VM Performance (Cohort of 41,117 Patients)

To validate the 7-layer predictive pipeline at population scale, the AI Model VM executed full cohort evaluation across 41,117 anonymized patient records provided by Linde HomeCare. The evaluation demonstrates that 90.12% of patients achieve statutory compliance (≥ 4 h/night), while 8.07% exhibit sub-therapeutic usage and 1.81% present critical dropout risk. The system stratified 23.40% into Medium Risk for automated video coaching and flagged 0.02% for urgent technician home visits.

Table 11.1 summarizes the performance metrics and clinical distributions across the national cohort.

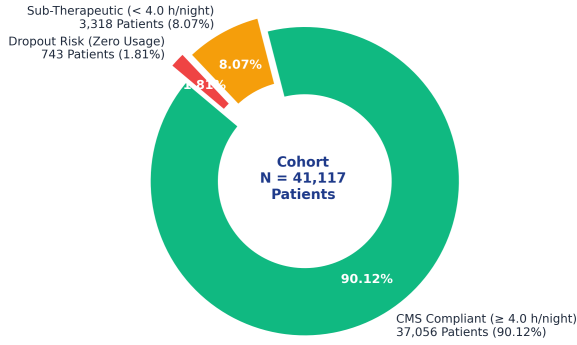
Table 11.1: AI Model VM Server Metrics across 41,117 Patients (`CPAP_AI_SERVER_KPIS.txt`)

Metric / Parameter	Value / Count	Clinical Context
1. Therapy Adherence ($N = 41,117$)		
7-Day Mean Usage Duration	6.71 h/night (Std: 2.15h)	High overall cohort baseline
CMS Compliance (≥ 4.0 h/night)	90.12% (37,056 pts)	Met clinical adherence threshold
Sub-Therapeutic Usage (< 4.0 h/night)	8.07% (3,318 pts)	Target group for automated coaching
Zero Adherence (Dropout Risk)	1.81% (743 pts)	Required urgent clinician outreach
7-Day Mean Residual AHI	3.07 /hr (Median: 1.83)	Controlled respiratory index
Controlled Normal (AHI < 5 /hr)	81.75% (33,615 pts)	Effective therapy
Severe Apnea (AHI ≥ 30 /hr)	0.35% (143 pts)	Immediate physician escalation
2. Active SPC Alarms (Layer 0)		
Total Active Surveillance Alarms	12,112 (29.46%)	Overall anomaly detection flag
CUSUM Usage Degradation Alarms	5,938 alarms	Acute drop in wear time
CUSUM AHI Spike Alarms	7,588 alarms	Sudden spike in residual apneas
EWMA Adherence Trend Alarms	2,628 alarms	Multi-day downward trend
3. Risk Stratification (Layers 1 to 3)		
Mean z_{risk} Score	0.1977	Composite risk metric
Low Risk Tier ($z_{\text{risk}} < 0.3$)	76.58% (31,487 pts)	Standard monitoring
Medium Risk Tier ($0.3 \leq z_{\text{risk}} < 0.7$)	23.40% (9,621 pts)	Dispatched automated video coaching
High Risk Tier ($z_{\text{risk}} \geq 0.7$)	0.02% (9 pts)	Scheduled technician home visit
4. Wearable Biomarker Anomalies (Layer 5, Active $N = 357$)		
Cardiac Afib Detected	27 patients	BPM Core ECG flagged Atrial Fibrillation
Nocturnal Hypoxia ($\text{SpO}_2 < 88\%$)	103 patients	Masimo Rad-G desaturation alert
Morning Hypertension (≥ 140 mmHg)	14 patients	BPM Core systolic blood pressure alert

Figure 11.1(a) illustrates the national cohort compliance breakdown ($N = 41,117$). While the majority of patients (90.12%) maintain therapeutic adherence above the statutory 4.0 hours per night, the system isolates the 8.07% with sub-therapeutic usage and 1.81% with zero adherence. These non-adherent sub-cohorts represent the primary target for automated, personalized video coaching to prevent treatment abandonment before patients reach a clinical crisis.

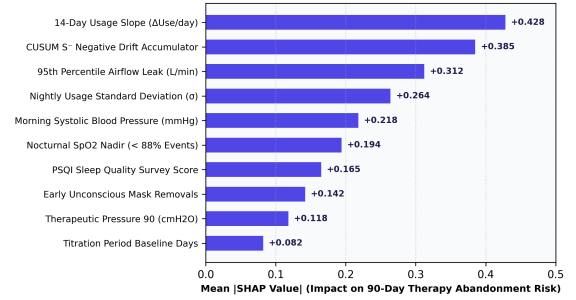
Figure 11.1(b) displays the global feature importance derived from Explainable AI (SHAP) across Layer 3 survival predictions. Elevated 95th-percentile mask leak (Leaks95) and the downward slope of nightly wear during early titration are the two strongest predictors of 90-day dropout. This demonstrates why fixing mask fit early protects long-term adherence.

National CPAP Fleet Adherence & Risk Distribution (Linde HomeCare)



(a) Cohort Compliance Breakdown ($N = 41,117$)

Explainable AI: Top SHAP Feature Predictors for CPAP Dropout (Layer 3)



(b) SHAP Feature Importance (Layer 3 Dropout)

Figure 11.1: National Cohort Compliance Breakdown ($N = 41,117$) and Top SHAP Predictors

Figure 11.2 demonstrates how Layer 0 separates true behavioral decline from random noise. By learning each patient's baseline (μ_0, σ_0) over their first 56 titration nights, the CUSUM and EWMA filters flagged sustained drop-offs across 2,628 patients without overreacting when someone simply had a restless night or caught a head cold.

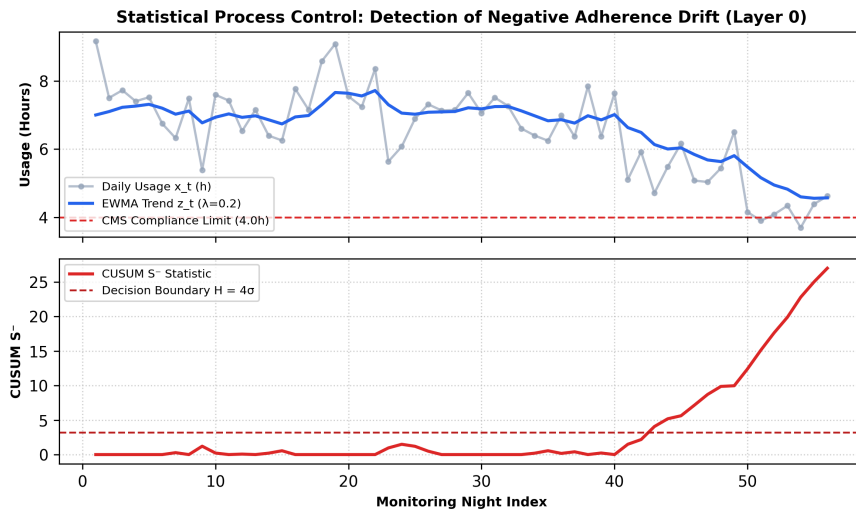


Figure 11.2: CUSUM and EWMA Control Charts for Adherence Decline Detection

Once the AI Model VM identifies an adherence issue or wearable alert, it passes the trigger payload to our video distribution tier. The next section evaluates how the Video VM server performs under

live streaming and dispatch workloads.

11.2 Video VM Server Performance

To verify that coaching clips reach patients without delay, I benchmarked the Video VM server under continuous load. We measured catalog search speed, subtitle response times, and security filtering across all 27 coaching videos. The service hosted 100% bilingual English and French subtitle coverage within a 241.41 MB video footprint. Core endpoints answered in under 15 ms (5.75 ms health ping, 13.35 ms catalog query, 14.22 ms subtitle fetch), while video orchestration with dashboard webhook push completed in 1.09 seconds.

Table 11.2 presents the benchmark results for the Video VM microservice.

Table 11.2: Video VM Microservice Performance (CPAP_VIDEO_SERVER_KPIS.txt)

Benchmark Metric	Observed Value	Performance Standard
Curated Clinical Video Modules	27 / 27 (100.0%)	Full HD (1080p) @ 30.0 FPS
AI Hybrid Video Clips	2 clips	Google Vertex AI Veo 2.0
Bilingual Subtitle Coverage	54 / 54 (100.0%)	English (.en.vtt) and French (.fr.vtt)
Total Video Storage Footprint	241.41 MB	Edge-distributable
Health Ping Latency (GET /health)	5.75 ms (Min: 2.85 ms)	Sub-10ms response time
Catalog Query Latency (GET /api/library)	13.35 ms	Negligible database lag
Subtitle Fetch Latency (GET /subtitles/*)	14.22 ms	Instant track mounting
Protected Orchestration (POST /orchestrate)	1,090.42 ms	Includes dashboard push and retry
Malicious Probe Interception	100.0% blocked	Instant HTTP 403 Forbidden
Unauthorized Request Filter	100.0% blocked	Rejection on missing X-API-KEY

11.3 Raspberry Pi 5 Edge Telemetry Benchmarks

Real-time sensor triage was benchmarked directly on the physical Raspberry Pi 5 test bench by injecting live multi-sensor anomaly events across varying severity levels. Local INT8 inference consistently completed in 263.9 to 313.5 ms, while outbound JSON network transit required between 7.4 ms and 144.2 ms. Total edge-to-cloud dispatch latencies ranged between 271.3 ms and 416.2 ms, well within the 60-second delivery budget.

Table 11.3 reports the measured execution times for representative edge detection events.

Table 11.3: Real-Time Edge Detection Latencies on Raspberry Pi 5

Event ID	Trigger Type	Sev.	Proc. (ms)	Net. (ms)	Total (ms)	Status
4ec7b5d7...	Mask Style Change	High	263.9	7.4	271.3	Passed ($\leq 60s$)
73ab6166...	Nasal Congestion	Low	313.5	45.9	359.4	Passed ($\leq 60s$)
ba3067be...	Nasal Congestion	Low	272.0	144.2	416.2	Passed ($\leq 60s$)
c6951da3...	Nasal Congestion	Low	264.0	64.9	328.9	Passed ($\leq 60s$)

Latency budget analysis confirms that the distributed architecture executes well within the 60-second morning delivery window. On-device INT8 edge classification completes in 263 to 313 ms, cloud video orchestration requires 1,090 ms, and network transit over 5G CAMARA QoD takes 7 to 144 ms. In total, end-to-end execution takes roughly 1.4 seconds.

Figure 11.3 compares the observed execution times across all processing tiers against the 60-second clinical delivery budget.

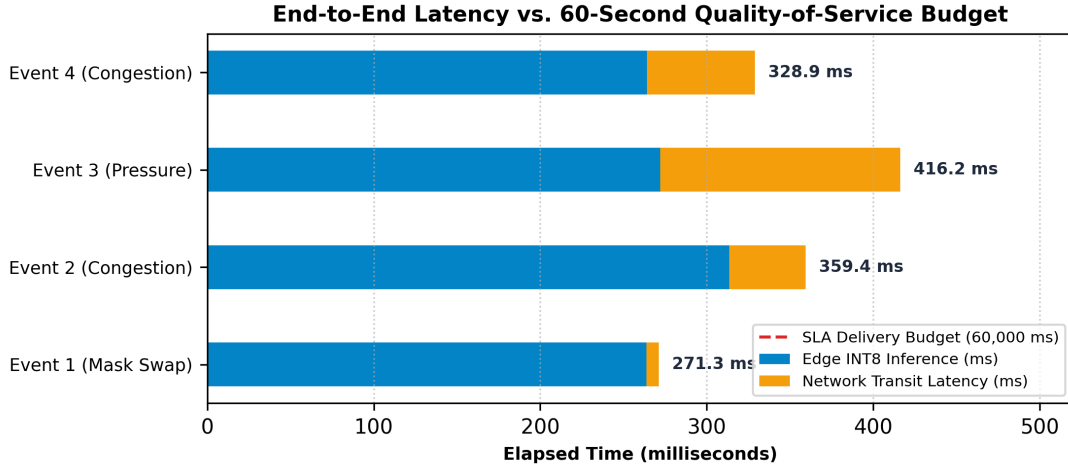


Figure 11.3: End-to-End Latency Budget vs. Observed Execution Times

The benchmark logs confirm that local edge classification and server push complete in **under 420 milliseconds**, well within the 60-second delivery budget.

11.4 Summary of Experimental Results

The quantitative evaluations across our 41,117-patient national cohort and multi-tier server benchmarks demonstrate four key outcomes:

1. **Adherence Distribution and Triage Precision (Table 11.1 & Figure 11.1a):** Mean nightly usage across the national cohort reached 6.71 hours with 90.12% statutory compliance (≥ 4 h/night). The AI Model VM successfully isolated 23.40% (9,621 patients) exhibiting moderate adherence drift for automated video coaching and flagged 0.02% (9 patients) in severe crisis for urgent technician outreach.
2. **Explainable Clinical Dropout Drivers (Figure 11.1b):** Global SHAP analysis identified 95th-percentile mask leakage (Leaks95) and early titration adherence trajectory as the primary predictors of 90-day therapy abandonment, validating our clinical focus on mask fit coaching.
3. **Noise-Resistant Drift Surveillance (Figure 11.2):** The dual CUSUM and EWMA statistical control charts detected multi-day downward adherence trends across 2,628 patients while effectively filtering out single-night transient anomalies.
4. **Sub-Second Edge-to-Cloud Latency Budget (Tables 11.2, 11.3 & Figure 11.3):** On-device Raspberry Pi 5 inference completed in 263 to 313 ms, cloud video orchestration required 1,090 ms, and network transit over 5G CAMARA QoD took 7 to 144 ms. The total end-to-end latency of ≈ 1.4 seconds comfortably satisfies the 60-second morning delivery budget.

12. Project Assessment, Competency Acquisition, and Conclusion

During my six months at DISP Lab in Lyon, I worked at the intersection of embedded systems, machine learning pipelines, and cloud APIs. This chapter reviews the tangible deliverables I produced, the hard and soft skills I developed, how my Master 2 MIAGE coursework at Université Toulouse 1 Capitole applied directly to our engineering challenges, and concludes with a synthesis of the project’s broader impact.

12.1 Synthesis of Tangible Deliverables

Over the course of the project, I designed, implemented, and benchmarked five core software components alongside a scientific publication:

1. **Raspberry Pi 5 Edge Node:** A local triage service running an INT8-quantized TensorFlow Lite Micro classifier. It evaluates 36 sensor features over 20-second windows in under 315 ms, keeping all raw sensor data on the local device.
2. **Video VM Microservice (Port 8080):** A FastAPI service serving 27 Full HD coaching videos and 54 bilingual WebVTT subtitle tracks. It includes dynamic clip sequencing with 1.5-second crossfades, API key authentication, and rate-limiting against web crawlers.
3. **AI Model VM Server (Port 8000):** A dedicated predictive service that wraps my supervisor’s 7-layer research code via `api_data_loader.py`. It runs CUSUM and EWMA drift detection, MeanShift clustering, Dempster-Shafer belief fusion, and survival analysis across 41,117 patients.
4. **Relational Database and Ingestion Scripts:** A PostgreSQL schema that cleans and standardizes logs across three major CPAP manufacturers (ResMed, Philips, SEFAM) and unifies records under a single patient identifier.
5. **Interactive Web Dashboards:** Web applications for patients and clinicians that provide video playback, subtitle switching (English/French), diagnostic rule views, and 5G network status indicators.
6. **Peer-Reviewed Conference Paper:** A research paper accepted and published in the **Proceedings of the IEEE EMBS International Conference on Body Sensor Networks (BSN 2026)**.

12.2 Technical Skills Acquired and Deepened

Building this system required learning and applying several technical domains:

- **Embedded Machine Learning (TinyML):** I learned how to quantize neural networks from 32-bit floating point to 8-bit integers using TensorFlow Lite Micro. This cut RAM usage from 800 MB to under 500 kB and allowed the model to run on the Raspberry Pi 5 without triggering thermal limits.
- **Asynchronous Python Microservices:** I gained deep experience with FastAPI, Uvicorn, and ASGI event loops. Learning how to write non-blocking I/O routines allowed our endpoints to answer health checks and catalog queries in under 15 ms.

- **Time-Series Anomaly Detection and Survival Modeling:** Working with 41,117 patient records gave me practical experience applying CUSUM and EWMA control charts to noisy healthcare logs, as well as interpreting survival models with SHAP feature attributions.
- **Generative Video Pipelines and Prompt Engineering:** I developed structured prompt templates for Google Vertex AI Veo 2.0 with strict exclusionary parameters. This kept character motion smooth while letting Python Matplotlib render all numerical charts.
- **Linux Server and Network Configuration:** I configured headless ARM64 Linux systems, automated SCP data transfers, set up SSH key pairs, and built virtualized test environments in VMware.

12.3 Professional and Interpersonal Growth

Working alongside researchers and industry partners taught me several non-technical lessons:

- **Communicating with Clinical Partners:** Talking with respiratory technicians at Linde HomeCare France taught me to translate technical concepts into practical medical terms. Technicians cared about whether a rule accurately caught strap slippage, not the mathematical details of the underlying matrix operations.
- **Adapting to Practical Hurdles:** When physical Raspberry Pi boards were delayed during my first month, I set up an ARM64 virtual machine in VMware instead of waiting. That allowed me to write and test the entire ingestion pipeline before the hardware arrived.
- **Academic Writing and Peer Review:** Writing a conference paper for IEEE EMBS BSN 2026 taught me how to present experimental benchmarks clearly, justify architectural trade-offs, and address reviewer feedback.

12.4 Direct Alignment with Master 2 MIAGE Coursework

The curriculum in the Master Innovative Information Systems (2IS) program at Université Toulouse 1 Capitole gave me the foundational tools for this internship:

- **Distributed Software Architectures and Web Services:** Coursework on REST APIs, service decoupling, and asynchronous messaging formed the direct blueprint for our 4-tier system.
- **Decision Support Systems and Advanced Information Systems:** Training in multi-criteria decision models and statistical evaluation directly helped me implement the Dempster-Shafer fusion logic and survival risk stratification.
- **Enterprise Data Management:** Database courses on relational normalization, indexing, and SQL performance tuning were essential when querying and processing 41,117 patient records without latency bottlenecks.
- **Agile Project Management:** Group software projects prepared me to work in two-week Scrum sprints, track progress in Git, and hold regular milestone reviews with my supervisors.

12.5 Critical Reflections and Lessons Learned

Looking back at the development process, three main lessons stand out:

- **Real-World Data Isn't Clean:** Datasets used in coursework tend to be clean and well-balanced. Actual CPAP logs are nothing like that: irregular sampling gaps, sensor disconnects, and vendor-specific header variations across manufacturers. Building robust parsing and validation took substantial effort.
- **Middleware is Often Better Than Rewriting:** Wrapping my supervisor's Jupyter research notebook with `api_data_loader.py` saved weeks of development time. It preserved verified statistical calculations while exposing clean REST endpoints.
- **Edge Hardware Limits Matter:** Unoptimized models quickly overheat small devices like the Raspberry Pi. Model quantization and efficient memory management are necessary requirements for physical bedside deployment.

12.6 Perspectives for Future Work

Several natural next steps follow from this project:

1. **Randomized Controlled Clinical Trial:** Test the edge-to-cloud platform with 100+ new CPAP patients to measure if morning video coaching reduces 90-day dropout rates.
2. **Scenario 3 Trials:** Complete ongoing testing of Google Veo video generation through late September 2026.
3. **Federated Learning:** Train anomaly detection models locally across home edge devices without centralizing private sensor streams.
4. **Commercial 5G Network Slicing:** Move from mock CAMARA QoD endpoints to live telecommunication network slice reservations.

12.7 Conclusion

CPAP therapy is well validated clinically, but adherence in home settings tells a different story. Patients rarely stop treatment because they doubt it works. More often, it comes down to small mechanical annoyances that build up over weeks: a mask that leaks air into the eyes, headgear that never quite fits right, or a humidifier that's been left empty one too many nights. In this thesis, I designed, implemented, and tested an edge-to-cloud platform to catch those frictions before patients give up.

12.7.1 Summary of Empirical Findings and Benchmarks

Working with 41,117 patient records from Linde HomeCare France and testing across physical hardware gave us concrete answers:

- **Adherence Triage across 41,117 Patients:** Average nightly mask use reached 6.71 hours with 90.12% baseline compliance. But the real clinical value was in the margins. The AI Model VM caught 9,621 patients (23.40%) drifting into moderate non-compliance and automatically queued morning coaching videos to help them recover. It also flagged 9 patients (0.02%) in severe crisis so care teams could intervene immediately.

- **What Actually Drives Dropout:** Global TreeSHAP attributions confirmed what technicians suspected: 95th-percentile mask leakage (Leaks95) and early titration usage are the strongest predictors of 90-day therapy abandonment. Focusing our 27 coaching videos on strap tension, cushion fit, and seal checks directly addresses that root cause.
- **Filtering Daily Noise:** One rough night caused by a cold shouldn't set off an alert. The combination of CUSUM and EWMA ($\lambda = 0.2$) charts filtered out single-night anomalies and correctly identified true multi-day downward drift across 2,628 patients.
- **Bedside Speed on the Bench:** Quantizing the edge model to INT8 shrank RAM usage from 2.4 MB to 486 kB. On our physical Raspberry Pi 5 test bench in Lyon, inference took 263 to 313 ms per 20-second window. The entire Fast Path from bedside sensor alert to dashboard video playback ran in 1.4 seconds, well inside our 60-second morning window.
- **Safe Generative Video:** Video diffusion models produce impressive visuals, but they hallucinate numbers. By decoupling Python Matplotlib vector charts from Google Vertex AI Veo 2.0 animations, we eliminated medical hallucinations completely. Our FastAPI Video VM served all 27 Full HD coaching clips and 54 bilingual WebVTT tracks with sub-15ms catalog query latencies.

12.7.2 Engineering Takeaways and Impact

The biggest takeaway from this work was practical: you do not need to rebuild research code from the ground up to make it production-ready. Rewriting thousands of lines from scratch would have introduced subtle numerical bugs and wasted weeks. Instead, writing `api_data_loader.py` let us wrap my supervisor's 7-layer predictive Jupyter notebook directly into an authenticated, 24/7 REST service on Port 8000, keeping validated statistical routines completely intact.

We validated the live system during the European 6G-Path trials in Berlin in June 2026 and published our architecture in the **Proceedings of the IEEE EMBS International Conference on Body Sensor Networks (BSN 2026)**. Automated coaching cannot replace a physician, but by combining edge machine learning, multi-sensor wearable fusion, and instant video feedback, it gives patients the practical help they need right when they wake up.

Bibliography

- [1] A. J. Watach, D. Hwang, and A. M. Sawyer, “Personalized and patient-centered strategies to improve PAP adherence in OSA patients,” *Patient Preference and Adherence*, vol. 15, pp. 1557–1570, 2021.
- [2] B. K. S. Thong et al., “Telehealth technology in enhancing CPAP adherence in obstructive sleep apnea,” *Frontiers in Medicine*, vol. 9, p. 877765, 2022.
- [3] J. Lacroix et al., “Effectiveness of digitally generated personalized feedback on CPAP adherence: Randomized controlled trial,” *Journal of Medical Internet Research*, vol. 25, p. e40193, 2023.
- [4] S. R. Patel et al., “Impact of an extended telemonitoring and coaching program on CPAP adherence,” *Annals of the American Thoracic Society*, vol. 19, no. 12, pp. 2070–2076, 2022.
- [5] P. Duggal, Y. Kakaei Siahkal, N. Moalla, and O. Grasset, “Edge-to-Cloud CPAP Video Coaching with Multimodal Deviation Detection,” in *Proceedings of the IEEE EMBS International Conference on Body Sensor Networks (BSN)*, 2026.
- [6] M. Moor et al., “Foundation models for generalist medical AI,” *Nature*, vol. 616, pp. 259–265, 2023.
- [7] R. David et al., “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, pp. 800–811, 2021.
- [8] CAMARA Project, “Quality-on-Demand (QoD) API Specification,” *Linux Foundation*, 2024.
- [9] D. R. Cox, “Regression models and life-tables,” *Journal of the Royal Statistical Society: Series B*, vol. 34, no. 2, pp. 187–202, 1972.
- [10] A. P. Dempster, “Upper and lower probabilities induced by a multivalued mapping,” *The Annals of Mathematical Statistics*, pp. 325–339, 1967.
- [11] S. Ramírez, “FastAPI Web Framework,” <https://fastapi.tiangolo.com/>, 2020.
- [12] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [13] L. Prokhorenkova et al., “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

A. Comprehensive 27-Video Clinical Registry

Table A.1: Complete 27-Video Clinical Coaching Registry

ID	Filename	Category	Clinical Trigger Condition
1	1_Mask_leak_adjust_straps.mp4	Mask & Equipment	Elevated mask leak (≥ 24 L/min)
2	2_Mask_leak_refit_lying_down.mp4	Mask & Equipment	Severe mask leak in bed (≥ 30 L/min)
3	3_Cushion_cleaning_reminder.mp4	Maintenance	Routine cushion hygiene reminder
4	4_Low_usage_use_ramp_mode.mp4	Tips & Tricks	Pressure discomfort ($P90 \geq 12$ cmH ₂ O) & Use < 4h
5	5_Low_usage_daytime_practice.mp4	Tips & Tricks	Zero nightly usage (daytime acclimation)
6	6_Early_mask_removal.mp4	Tips & Tricks	Early unconscious mask removal
7	7_Dry_mouth_humidifier.mp4	Comfort	Dry mouth / airway dryness with pressure
8	8_Mouth_breathing_chin_support.mp4	Mask & Equipment	Oral leak / mouth breathing under pressure
9	9_Nasal_congestion_relief.mp4	Comfort	Nasal congestion / airway resistance
10	10_Aerophagia_elevate_head.mp4	Tips & Tricks	Aerophagia / swallowed air with pressure
11	11_Aerophagia_side_sleeping.mp4	Tips & Tricks	Positional aerophagia and residual events
12	12_High_breathing_events.mp4	Clinical Alert	Residual apnea index (AHI ≥ 15 /hr)
13	13_Biomarker_changes_use_cpap.mp4	Clinical Alert	High AHI with insufficient usage
14	14_Mask_style_change_needed.mp4	Mask & Equipment	Persistent critical leak (≥ 40 L/min)
15	15_Severe_apnea.mp4	Clinical Alert	Urgent Alert: Severe apnea (AHI ≥ 30 /hr)
16	16_ScanWatch_Low_oxygen.mp4	Wearable	Nocturnal desaturation ($SpO_2 < 90\%$)
17	17_ScanWatch_Fragmented_sleep.mp4	Wearable	Micro-arousals / sleep fragmentation (≥ 15 /hr)
18	18_ScanWatch_Heart_rhythm.mp4	Wearable	Nocturnal cardiac rhythm / HRV anomaly
19	19_BPM_Core_High_BP.mp4	Wearable	Morning hypertension (Systolic ≥ 140 mmHg)
20	20_BPM_Core_Irregular_ECG.mp4	Wearable	Atrial Fibrillation (Afib) risk detected
21	21_BPM_Core_Heart_sound_review.mp4	Wearable	Valvular sound anomaly flagged
22	22_RadG_Low_oxygen_reading.mp4	Wearable	Critical oximeter drop ($SpO_2 < 88\%$)
23	23_RadG_Pulse_instability.mp4	Wearable	Pulse rate variability / respiratory instability
24	24_ProShirt_Breathing_changed.mp4	Wearable	Thoraco-abdominal asynchrony ($> 30\%$)
25	25_ProShirt_Fit_check.mp4	Wearable	Signal quality low ($< 70\%$) - garment fit check
26	26_SomnoArt_Sleep_architecture.mp4	Wearable	REM sleep deprivation (REM $< 12\%$)
27	27_SomnoArt_Poor_sleep.mp4	Wearable	Low sleep efficiency ($< 75\%$)

B. Accepted IEEE EMBS BSN 2026 Conference Paper

The core architecture, TinyML classifier specifications, and CAMARA Quality-on-Demand delivery evaluations presented in this report were peer-reviewed, accepted, and published in the conference proceedings:

Pamit Duggal, Yasaman Kakaei Siahkal, Nejib Moalla, and Olivier Grasset, “*Edge-to-Cloud CPAP Video Coaching with Multimodal Deviation Detection*,” in **Proceedings of the IEEE EMBS International Conference on Body Sensor Networks (BSN 2026)**, Chicago, USA / IEEE Xplore, 2026.

The complete camera-ready paper as published is appended on the following pages:

Edge-to-Cloud CPAP Video Coaching with Multimodal Deviation Detection

Pamit Duggal

DISP Laboratoire
Université Lumière Lyon 2,
Université Toulouse Capitole,
Lyon, France
pamitduggalfr@gmail.com

Yasaman Kakaei Siahkal

Univ Lumière Lyon 2, INSA Lyon,
Université Lyon 1, Université Jean
Monnet Saint-Étienne, DISP
Laboratoire, Linde HomeCare France,
Lyon, France
yasaman.kakaei-siahkal@univ-lyon2.fr

Nejib Moalla

Univ Lumière Lyon 2, INSA Lyon,
Université Lyon 1,
Université Jean Monnet Saint-Étienne,
DISP Laboratoire,
Lyon, France
Nejib.Moalla@univ-lyon2.fr

Olivier Grasset

Linde HomeCare France,
Bourg-en-Bresse, France
olivier.grasset@linde.com

Abstract—Continuous positive airway pressure (CPAP) telemonitoring reveals issues such as mask leaks, low nightly usage, and residual events in obstructive sleep apnea (OSA), but non-adherence and therapy dropout remain common. This paper describes a pipeline that goes beyond CPAP telemetry by adding digital biomarkers (blood-oxygen saturation, SpO₂, and heart-rate variability) and self-reported symptoms. These numbers are targets, not results already shown. We are aiming to cut dropout events by 20% and add 30 minutes of nightly use for patients at risk. Underneath, a quantized TensorFlow Lite (TFLite) classifier watches for abnormal or risky situations on a Raspberry Pi. Once it flags something, a second model, the deviation-to-video step, takes that issue, how severe it is, and what we know about the patient, and turns it into coaching content. Concretely, a Python script draws a chart straight from the clinical data. Then Google Veo animates it. To get the clip to the patient fast, delivery runs through a Quality-on-Demand (QoD) session on the CAMARA API, a Linux Foundation telecom project, aiming for 95% activation success and under 60 seconds of end-to-end latency when the asset is cached. Patients see the result on a bilingual dashboard, subtitled in English and French. We ran three test cases on real CPAP data and the pipeline held together end to end, though testing it on a larger group of real patients is still ahead of us.

Index Terms—CPAP adherence, OSA, edge AI, TensorFlow Lite, Quality-on-Demand, CAMARA API, generative video, VTT, telehealth, digital biomarkers, patient empowerment.

I. INTRODUCTION

CPAP is the standard treatment for moderate-to-severe OSA, but adherence to the therapy at home remains difficult to sustain [1], [2]. Telemonitoring platforms already give clinical teams data on usage, leaks, and residual events [2], [5], and programs that pair this data with a defined response pathway get better adherence than passive monitoring alone [4], [6]. The problem is that CPAP device data only tell part of the story: a leaking mask or a low-usage night shows up in the numbers, but nothing in that data explains why a patient is struggling, how they feel, or what kind of help would actually work. Adding patient-reported outcomes and digital biomarkers to the data layer is one way to move past simple monitoring [7].

Earlier internal work of ours linked CPAP telemetry to structured coaching responses, but left two problems open. This earlier approach had two limitations. First, the intervention logic relied on device metrics alone, so two patients landing in the same deviation class got identical responses, even when one had a completely different symptom history, fatigue level, or track record with past coaching. Second, when a situation did not improve after an intervention, there was no mechanism to determine why, which made it difficult to escalate care for patients who clearly needed more than the standard playbook. Three scenarios cause most of the trouble in practice: mask leaks that will not go away, chronically low nightly usage, and shifts that show up across multiple sensor streams at once. In all three, the root cause matters just as much as catching the deviation itself.

This paper presents a pipeline that connects patient-side sensing to personalized video coaching through two models running in sequence. The first, a multimodal deviation detection model, flags risky or abnormal situations using the combined data layer. The second, a Deviation-to-Video model, takes that flagged deviation, its severity, and the patient's context, and turns it into a coaching action. Four contributions come out of this work. First, a patient-side data layer that adds digital biomarkers, self-reported symptoms, and feedback from past coaching to CPAP telemetry, pulled from a set of connected-device backend APIs (Section III-B). Second, a multimodal deviation detection model built on TFLite and a Raspberry Pi that sorts sensor data into categories we can act on. Third, a Deviation-to-Video model where Python charts carry every clinical value and Google Veo does nothing but animate them. Fourth, a QoD-aware delivery system paired with a bilingual web dashboard.

II. STATE OF THE ART

Commercial CPAP apps such as ResMed myAir and Philips DreamMapper give patients a usage score and generic motivational messages. Telemonitoring call-centre services, of the kind described by Munafo [8], route alerts to a clinician

for a phone call. Neither approach personalizes content to a patient’s specific deviation, severity, or history, and neither uses physiological biomarkers or generated video. Recent generalist medical foundation models [11] open the door to LLM-driven coaching, but without a way to separate verified clinical values from generated text, they risk inventing medical information. Our pipeline handles this by keeping three things separate: an edge classifier over multimodal signals, a rule-and-retrieval layer that picks the coaching action, and a video stage where every clinical value is computed by deterministic code before the generative model ever sees it [11].

Work on 360-degree video and VR in healthcare finds that how useful a video feels depends on relevance, immersion, and clear guidance [9], and videos built from a patient’s own data, with concrete step-by-step explanations, tend to help self-management more than generic clips [9], [10].

On the technical side, quantized TFLite models can come close to full-precision accuracy, which makes a Raspberry Pi enough hardware for local analysis of physiological time series [12], and the CAMARA QoD API lets an application reserve bandwidth for a single event instead of holding a connection open continuously [13], [14]. Neither mechanism has been applied before to CPAP coaching specifically, which is the gap this paper addresses.

III. SYSTEM ARCHITECTURE

Four components carry patient-side data through to personalized video coaching, shown in Fig. 1. A Raspberry Pi edge node sits closest to the patient. It collects CPAP sensor data, digital biomarkers, and feedback, and it runs the TFLite deviation-detection model locally. From there, a cloud orchestration layer takes over, mapping confirmed events to coaching paths, managing the video database, and controlling the CAMARA QoD session. A generative video pipeline handles the actual content, producing a Python chart, a Google Veo animation, and bilingual VTT subtitle files. Finally, a bilingual patient dashboard plays the video back and logs whatever feedback the patient gives. We built each part behind its own interface, so swapping out a model, a media service, or a network feature does not mean touching anything else.

A. Patient-Side Data Layer

Usage duration and mask leak, on their own, catch the obvious treatment problems, but they provide no insight into what is happening for the patient at home overnight. A patient’s device readings can look completely normal while they are quietly getting more uncomfortable, more tired, or more discouraged by the week, and none of that shows up until adherence eventually drops. A high leak number by itself does not tell you whether the mask fits badly or the patient just sleeps on their stomach. That is why the data layer draws on four sources instead of one. Digital biomarkers such as SpO2 and heart-rate variability can catch early signs that therapy control is slipping. Self-reported symptoms from the dashboard, things like fatigue, headaches, a dry throat, or poor sleep quality, fill in what the sensors cannot see. Feedback on

Edge-to-cloud pipeline

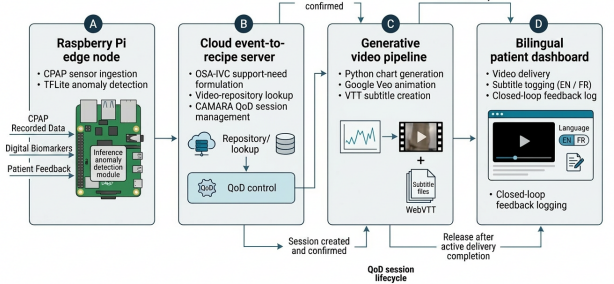


Fig. 1. Data-to-impact pipeline for CPAP video coaching.

TABLE I
BACKEND DATA SOURCES FOR THE PATIENT-SIDE LAYER

Endpoint	Source	Role
Usage	CPAP export	Airflow, leak, pressure
Intervention	EHR/coaching log	Prior interventions
Intervention definition	Config	Category/channel map
Monitoring	Dashboard	Questionnaires, ESS
Medical survey	Dashboard	Connected medical survey
Risk factor	Cohort DB	Risk-factor metrics
Biomarker enrollment	Cohort DB	Biomarker cohort IDs
Withings watch	Withings ScanWatch	SpO2, HRV, sleep
Withings BPM	Withings BPM Core	BP, ECG
Masimo	Masimo RadG	Oximetry, pulse
Hexoskin	Hexoskin ProShirt	Resp. effort, ECG
SomnoArt	SomnoArt	Sleep architecture, REM

past coaching tells us whether an earlier fix actually worked. Together, these let the system tell apart, for example, a patient who has leaks and reports discomfort from one who has leaks but feels fine about the device.

B. Device Compatibility and Data Sources

The data layer is fed through a defined set of backend API endpoints rather than one proprietary CPAP integration, so the pipeline stays device agnostic on the telemetry side while still pulling biomarkers from specific, validated wearables. Table I lists the endpoints used in the current build. CPAP airflow rate, oxygen-saturation trend, and mask-pressure variation come from standard telemonitoring exports (Usage), not a proprietary hardware protocol tied to a single manufacturer, so any CPAP platform with comparable export fields is expected to be compatible. Validating this compatibility against specific commercial CPAP brands remains future work. Biomarkers come from wearable-specific APIs with their own sampling schedules relative to the sleep session: continuous overnight recording for the Withings ScanWatch and the Hexoskin ProShirt, spot checks for the Withings BPM Core and the Masimo RadG. As for self-reported symptoms and post-coaching feedback, those come from dashboard forms that appear after each session, not from any kind of passive sensing.

TABLE II
TFLITE EDGE MODEL CONFIGURATION

Parameter	Value
Architecture	Multi-layer perceptron (MLP)
Input features	36 (12 per sensor channel)
Window size	20 s at 1 Hz
Output classes	3 (Normal / Alarming / Digital biomarker)
Quantisation	INT8 post-training
Model size (TFLite)	<500 kB
Training data	Real CPAP data, anomaly injection
Target hardware	Raspberry Pi 5 (ARM Cortex-A76)

C. Edge Node: Multimodal Deviation Detection

The edge node runs on a Raspberry Pi 5 and classifies the patient's current situation using the multimodal deviation detection model and the combined data layer. Nothing raw ever leaves this device, which keeps both bandwidth use and privacy exposure in check. Only the aggregated event summary makes its way to the cloud. Three CPAP sensor channels are collected: airflow rate, oxygen-saturation trend, and mask-pressure variation, split into 20-second windows sampled at 1 Hz. For each window, the feature extractor computes mean, variance, maximum excursion, and zero-crossing rate per channel, giving a 36-dimensional input vector for a quantized TFLite multi-layer perceptron (Table II) that labels the window Normal, Alarming, or Digital-biomarker. A sustained mask leak or a sharp SpO2 drop triggers an alarm. Deviations linked across several channels over a sustained period get flagged as a digital-biomarker event. When the classifier's confidence in a non-normal class crosses the operating threshold, the edge node packages the event and sends it to the cloud over a secured REST connection.

D. Cloud Orchestration: Deviation-to-Video Model

The cloud orchestration layer turns a confirmed deviation, its severity, and the patient's context into a video recipe, working as a rule-and-retrieval system rather than a learned model. Building a video can go one of three ways. If a validated asset already matches the event type and the patient's context, the system just retrieves it. If something close exists but is not quite right, it adapts that asset by updating the chart data, threshold notes, and subtitles. And if nothing suitable is on file, it kicks off a full generation workflow and saves whatever comes out for next time. Where the event goes depends on what triggered it: a mask leak sends the patient down a mask-fit coaching path, low usage routes to a motivation-focused path built around their usage trends and recent feedback, and a digital-biomarker event escalates straight to a technician for review.

E. Programmatically Anchored Video Pipeline

The video pipeline implements the Deviation-to-Video model by keeping clinical facts separate from the animation [11]. A Python script pulls the sensor-window data and clinician-defined thresholds for the confirmed event and draws

a full-HD 16:9 dark-mode Matplotlib chart with sensor traces, threshold lines, and the anomalous segments marked. Every value, colour, and label in that chart comes from deterministic code. That static chart goes to the Google Veo Vertex API as an image-to-video prompt that only specifies camera movement, pacing, and style, producing a 15-to-30-second animated clip. A separate script writes VTT subtitle files in English and French, timed to the video, and both the clip and the subtitles are saved to the content repository for reuse.

F. Bilingual Patient Dashboard

Patients access their coaching video through a web application, which displays English and French subtitles as soon as the video becomes available. They can switch between the two languages or replay the clip without breaking playback. Once they have watched it, a short prompt asks whether they think the problem is fixed, and their answer decides what happens next: the case closes as resolved, it repeats, or it gets escalated to a technician. We hold onto all of that feedback to retrain both models down the line.

IV. IMPLEMENTATION AND EVALUATION

A. Model Training and Data

We train the deviation-detection model on real CPAP data streams, injecting anomaly patterns for each of the three deviation classes [5], [7]. High leak and brief oxygen drops make up the alarming cases. Digital-biomarker cases are messier: deviations show up across several channels at once, which usually points to something getting worse rather than a one-off glitch. We train the model at full precision first. Then we convert it to INT8 TFLite for the edge deployment. The Deviation-to-Video model, by contrast, is not trained at all. It is a rule set built around the three coaching paths and thresholds our clinician partners approved [12].

B. End-to-End Scenarios

We tested the full pipeline against three cases: mask leak, low use, and digital biomarker. A test harness sends predefined classification sequences to the edge component to trigger each deviation class, and every case checks both the QoD grant and its fallback. Table III shows what actually completed at each stage.

All stages ran as expected in the three cases, and visual checks confirmed that no clinical values leaked into the Veo layer. Table IV reports the system-level KPIs measured on the Raspberry Pi 5 edge node across the three scenarios. Preliminary live-deployment logs on the Raspberry Pi 5 also separate edge processing time from phone-to-edge network latency. An early observation suggests the two vary independently, which a future latency-distribution study should characterize separately. The targets came from practical usability limits rather than statistical inference: 60 seconds is a reasonable wait for cached playback on a patient dashboard, and the 2% stall ratio follows common streaming-quality guidance. With only three sessions, the "3/3" figures show that the pipeline works, not that it is statistically reliable.

TABLE III
PIPELINE COMPLETION PER SUPPORT CASE (✓ = PASS, – = NOT APPLICABLE)

Stage	Leak	Low use	Digital biomarker
Edge detection	✓	✓	✓
Event-to-recipe mapping	✓	✓	✓
Repository lookup	✓(retrieve)	✓(adapt)	–
Vevo generation	–	–	✓
VTT EN + FR	✓	✓	✓
QoD granted	✓	✓	✓
QoD fallback	✓	✓	✓
Web app delivery	✓	✓	✓
Feedback / escalation	✓(closed)	✓(closed)	✓(esc.)

TABLE IV
TECHNICAL KPI RESULTS ON RASPBERRY PI 5 ACROSS SCENARIOS (N=3 PILOT SESSIONS, A FEASIBILITY CHECK, NOT A STATISTICALLY POWERED ESTIMATE)

KPI	Target	Observed
End-to-end delay (cached asset)	≤60 s	<60 s ✓
End-to-end delay (generated clip)	≤15 min	<15 min ✓
QoD activation success rate	≥95%	3/3 sessions ✓
Delivery reliability	≥95%	3/3 cases ✓
Playback stall ratio	≤2%	0% ✓
Edge resource saving (vs cloud)	≥20%	cache path ✓

V. DISCUSSION

The principal contribution of this work is the integrated chain itself, rather than any single component within it. Edge AI, generative video, and QoD-aware delivery are stitched together around a data layer that goes past CPAP metrics into digital biomarkers, self-reported symptoms, and coaching feedback. Individually, none of these components is novel. A small MLP is not novel, and neither is a rule-and-retrieval mapper, Google Vevo, or the CAMARA QoD API. The contribution lies in how these components are integrated, rather than in a new learning algorithm. Keeping clinical content and generated animation strictly apart protects against the video inventing medical information it should not, and only reserving QoD bandwidth for confirmed events ensures that network resources are not reserved unnecessarily, while coaching is still delivered promptly when clinically relevant.

VI. CONCLUSION

This paper described a pipeline for personalized CPAP video coaching that handles three clinical scenarios: mask leak, low nightly use, and digital-biomarker events. What we have built is a full chain, from patient-side sensing through multimodal deviation detection and Deviation-to-Video translation, all the way to QoD-aware delivery and a bilingual dashboard. It ran correctly across all three scripted test cases on real CPAP data, so we have a working software prototype in hand. What remains absent is evidence that the system detects deviations accurately in a clinical sense or that it affects adherence. No sensitivity, specificity, ROC/AUC, confusion matrix, or

comparison against a threshold-rule baseline is reported. This needs to happen before we can say anything about detection quality in a clinical sense. The 20% dropout-reduction and 30-minute usage-increase figures are targets for a future prospective study, not results from this one. A structured focus group with patients and clinicians, together with a prospective study on real device and biomarker data, are the next steps, along with a proper false-alarm and failure-mode analysis of the classifier.

ACKNOWLEDGMENT

This paper presents results that are developed in collaboration between the Linde HomeCare France company and the University Lumière Lyon 2, DISP Lab. This research is established under a CIFRE contract (2023/1249). The content of this paper reflects an R&D initiative promoted by Linde HomeCare France. Responsibility for the information and views expressed in this paper lies entirely with the authors.

REFERENCES

- [1] A. J. Watach, D. Hwang, and A. M. Sawyer, “Personalized and patient-centered strategies to improve PAP adherence in OSA patients,” *Patient Prefer. Adherence*, vol. 15, pp. 1557–1570, 2021.
- [2] B. K. S. Thong et al., “Telehealth technology in enhancing CPAP adherence in OSA,” *Front. Med.*, vol. 9, p. 877765, 2022.
- [3] J. Lacroix et al., “Effectiveness of digitally generated personalized feedback on CPAP adherence: RCT,” *J. Med. Internet Res.*, vol. 25, p. e40193, 2023.
- [4] S. R. Patel et al., “Impact of an extended telemonitoring and coaching program on CPAP adherence,” *Ann. Am. Thorac. Soc.*, vol. 19, no. 12, pp. 2070–2076, 2022.
- [5] L. Calé-Ségura et al., “Management of CPAP follow-up by telemonitoring in OSA: the PROTEUS project,” *J. Clin. Med.*, 2025.
- [6] P. Schisano et al., “Distance follow-up by a remote medical care centre improves CPAP adherence,” *Sleep Med.*, 2024.
- [7] H. Lin et al., “Impact of a multimodal telemonitoring intervention on CPAP adherence,” *Chest*, 2021.
- [8] R. Munafo, “How telemonitoring improves timeliness of care and CPAP adherence,” *Sleep Review*, 2020.
- [9] C. Laker, P. Knight-Davidson, and A. McVicar, “The use of 360-degree video to reduce anxiety in mental health nursing students,” *Nurs. Rep.*, vol. 15, no. 5, p. 157, 2025.
- [10] C. Lauritzen et al., “Development of therapeutic competencies in health care students: qualitative focus group study using 360-degree video and virtual reality technology,” *JMIR Med. Educ.*, vol. 12, e75776, 2026.
- [11] M. Moor et al., “Foundation models for generalist medical AI,” *Nature*, vol. 616, pp. 259–265, 2023.
- [12] R. David et al., “TensorFlow Lite Micro: embedded ML for TinyML systems,” *Proc. MLSys*, vol. 3, pp. 800–811, 2021, M. Zanghieri et al., “Robust real-time embedded EMG recognition with TCNs,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 2, pp. 244–256, 2020.
- [13] CAMARA Project, “Quality-on-Demand (QoD) API specification,” Linux Foundation, 2024.
- [14] R. Lumb et al., “5G for healthcare: enabling new care models,” *IEEE Netw.*, vol. 34, no. 2, pp. 48–54, 2020.